

2023年

情報科学実験 I 指導書



A コンテンツ制作と音声処理



B 画像処理と生成



C 通信ネットワーク



D データ科学と自動制御



愛知県立大学 情報科学部

Aichi Prefectural University

C-1 通信基礎実験

「C-1 通信基礎実験」指導書の構成

章	内容
1	本実験の目的と注意事項
2	基礎知識の解説（ネットワークプログラミング，音声通信等）
3	実施する実験の内容（1日目，2日目，3日目に分かれている）
4	レポートで実施する「課題」
参考文献 付録	参考となる書籍 ※この他，インターネットで得られる情報も参考になる 主なシステムコールやコマンドの説明など

1 はじめに

1.1 本実験の目的

ネットワークを通じてデータ伝送を行うことを体験的に学ぶために，本実験では TCP/IP プロトコルを使い，ワークステーション間で通信を行うクライアントサーバモデルのプログラムを作成する．本実験では，基本的な文字情報とマルチメディアデータの一種である音声情報の伝送を体験する．

1.2 予習について

1.2.1 1日目開始前まで

指導書全体に目を通しておくこと．特に，3章の実験と4章の課題については3日目までの全体を事前に熟読し，1日目から3日目まで，課題も含めた流れを把握しておくように．2章から順に熟読するのも良いが，2章にざっと目を通してから3章を熟読し，疑問意識を持って2章を熟読して付録を参照するなどの方が効率的なこともある．用語やコマンドでわからないものがあれば，事前に参考文献やインターネットで調べておくことを勧める．調べた際に，調べた内容だけでなく，文献名や参照した URL についてもメモを取っておくと，レポート作成の効率化につながる．

本実験は演習室5で実施するが，基本的な環境は実験前にすでに整っており，ファイルも前もって置いてある．1日目の実験について，穴空きプログラムをコピーしプログラムを作成しておくことなどは，実験前でも一応可能である．個人差はあるが，穴空きプログラムの理解にもある程度の時間を要するので，予習の機会を有効に使うことが効果的である．

1.2.2 2日目および3日目開始前まで

2日目の実験では，自分の好きな機能を考え実行する．機能を考えることやプログラミングをしておくことは，2日目の実験前でも可能である．

3日目の実験についても，3日目の実験前に行えることが多い．実験を効率的に行うために有効に予習を実施して欲しい．

1.2.3 注意

1日目の実験までに，提供するファイルが更新されることもあるので，早めにスタートした人は更新が無かったかに注意するように．特に，2022年度は本実験の初年度であるため，1日目の実験の概ね1週間前までは提供ファイルが大きく変わる可能性がある．しかし，練習の意味で事前に行ってみることは，穴空きプログラム中のわからないコマンド等を調べておくことなどは無駄にはならないので，更新される可能性があることを理解の上で，積極的に予習することを勧める．

1.3 レポート（報告書）

1.3.1 レポート提出について

- レポート提出数：1人1通
- レポート提出〆切：3日目の実験の1週間後の12:50
- レポート提出方法：UNIPA（Universal Passport）の「課題管理」システムから提出
- レポート形式：A4サイズ、PDF または Word、 $\text{\LaTeX}$ で作成して PDF 形式で提出することを推奨。容量が6MB 以下となるようにすること。

1.3.2 レポート作成にあたって

3日分の実験に対し、1通のレポートを作成し提出することになる。実験終了して1週間後が提出期限であるため、全ての実験が終了してからレポート作成を開始しても間に合わない。最初の実験を開始する前に、実験目的や実験方法の下書きは書くことができ、1日目の実験終了後に1日目の実験方法の完成版、実験結果や考察、また1日目に関連する課題の部分は書くことができる。2日目の実験終了後にはレポートの3分の2以上の部分を完成させることができるはずである。このように、レポートは3週間かけて計画的に作成すること。

1.3.3 レポート記載項目に関する補足

基本は実施要項に記載の通り。以下補足。

- 実験方法
実験方法には、サーバ、クライアントに使用するワークステーションのホスト名も書くこと。
- 実験結果
実験結果には、すべてのソースプログラムのリストを添付し、プログラムの説明を加え、また、コンパイル方法、実行方法も記載すること。ただし、2日目の実験で複数種類のサーバプログラムを作成した場合は、2個目のソースプログラムからは1個目と異なる部分のみを書けば良い。その際、1個目のプログラムのどこを変えたかを明示すること（例えば、1個目のプログラムの該当部分について、行数を明記する、枠で囲う、色を変えるなど）。
プログラムの実行結果もわかりやすく、説明を付して書くこと。その際、1, 2日目の実験に関しては、送信した主な文字列とサーバから帰ってきた文字列を明記し、説明を加えること。3日目の実験に関しては、入力した音声についてレポートに文章で説明すること。

- 考察

考察には，課題として与えられた考察以外に，実験結果に対して自分で考察を行い，それをレポートに書くこと．

1.4 関連する科目など

通信，ネットワーク系の科目，プログラミングの科目などが，本実験に直接関連している．3年生や4年生向けに開講されている科目もあり，先に実験を体験した後，それらの科目を履修し，詳しい理論を高学年で学ぶということも有意義である．通信，ネットワーク系の科目としては，コンピュータネットワーク論（3年）と通信ネットワーク（3年）があり，プログラミングの科目としては，プログラミングⅠ（1年）からⅤ（3年）があるが，Ⅳ（2年）がもっとも関連している．通信に関しては，本実験で取り扱ったものよりもさらに基礎的あるいは原理的なことを学ぶ科目として，情報理論（2年），符号理論（2年），通信理論（3年）などがある．課題でも触れる通り，セキュリティとも関連が深いが，それは情報セキュリティ論（3年）で学ぶ．その他，本学部の多くの科目が直接的，間接的に関連している．

2 解説

以下，2.1と2.2では，3日間の実験で共通に必要な知識であるネットワーク及びネットワークプログラミングについて解説する．3日目にはさらに音声通信の知識が必要である．音声通信については2.3で解説する．

2.1 ネットワーク

2.1.1 ネットワークとプロトコル

ネットワーク（コンピュータネットワーク）とは2台以上のコンピュータが何らかの線でつながったものである．しかし，線で接続されているだけではコンピュータ間で通信を行うことができず，コンピュータ間の通信を行う物理的な手順（ネットワークカードなどのハードウェアに関する通信方法），論理的な手順（アプリケーションにおけるソフトウェアに関する通信方法）を与える必要がある．このようなコンピュータ間の通信手順の取り決めをプロトコルという．

インターネットとは，大学や企業などの組織内で構築されたネットワークを相互に接続する広域ネットワークで，インターネットプロトコル（IP）と呼ばれるプロトコルによって通信が行われる．通信線には電話線，イーサネットケーブル，同軸ケーブル，光ケーブル，鳩 [7][8] などがある．

2.1.2 プロトコルの階層モデル

通信を行うアプリケーションプログラムを作成するとき、通信の手順を定めたプロトコルを決定する必要がある。しかし、ネットワークインタフェースの電気的な特性などを考慮した、低レベルなプロトコルを考慮する必要はない。これは、プロトコルをレベルによって階層化し、必要なレベルのプロトコルだけに注目してアプリケーションを作成することができるからである。

プロトコルの概念的な階層モデルとして国際標準化機構（International Organization for Standardization, ISO）により定められた OSI（Open Systems Interconnection）参照モデル（図 1）が提供されている。

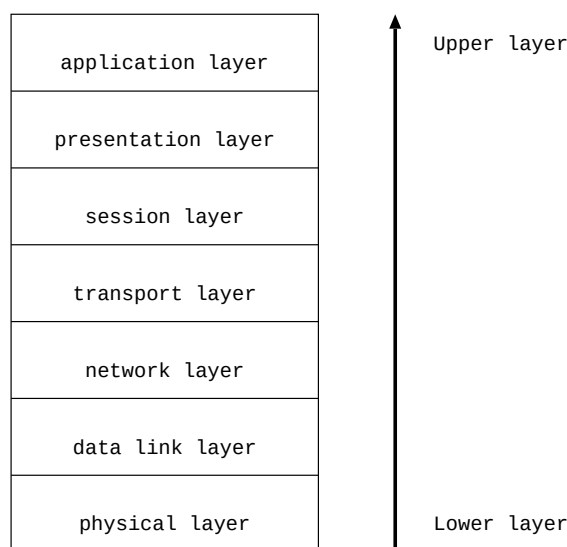


図 1: OSI reference model

OSI 参照モデルは 7 階層から成り立ち、物理媒体に近い方から物理層、データリンク層、ネットワーク層、トランスポート層、セッション層、プレゼンテーション層、アプリケーション層に別れている。

- 物理層
ネットワークを使用してデータを伝えるためのハードウェア（モデムやネットワークカード、ケーブルなど）の電氣的、物理的特性を規定する。
- データリンク層
ネットワークノード（コンピュータなどネットワークに接続されている機器）が物理層にアクセスする方法、データ形式、エラー訂正方法を規定する。
- ネットワーク層
ネットワーク上の 2 ノード間のデータ交換方法を規定するとともに、ネットワーク内の経路制御を行う。

- トランスポート層
2 ノード間での信頼性の高い連続データ転送手段を提供する。
- セッション層
通信相手ノードの選択やデータ形式などを対話的に行う方法を規定する。ユーザがネットワークを使用する場合に行われるコマンド入力やエラー表示がこれにあたる。
- プレゼンテーション層
アプリケーションプログラムがネットワークを使用するときのデータ形式を規定する。グラフィックイメージのデータ変換、テキストの圧縮などが提供されている。
- アプリケーション層
ユーザが直接操作するネットワークアプリケーションを規定する。

2.1.3 TCP/IP, UDP

TCP/IP はアメリカ国防総省の高等研究計画局（Advanced Research Projects Agency, ARPA）のネットワークである ARPANET 上で開発された。当初 ARPANET はパケット交換を使用して広域ネットワークを構築するために設計されたが、TCP/IP プロトコルは LAN にも適用されるようになった。

1980 年代始めに発表された UNIX 4.2BSD において TCP/IP が実装されていたことから、ワークステーションの通信プロトコルとして実質的な標準となり、1983 年には ARPANET の標準プロトコルとして使用されるようになった。

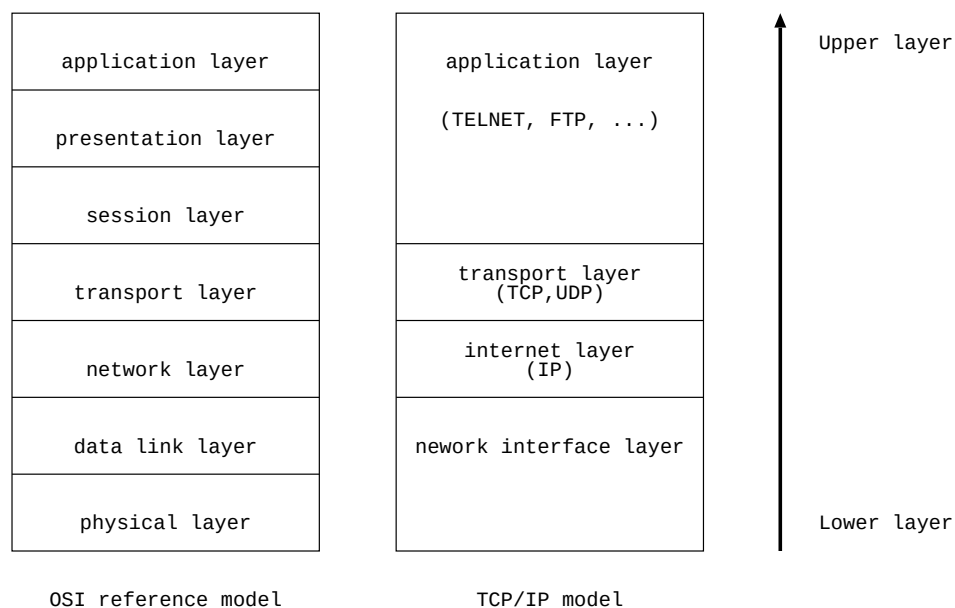


図 2: TCP/IP layer model

TCP/IP の階層モデル（図2）は，次の4つの階層から成り立っており，それぞれの階層はOSI 参照モデルのひとつまたはいくつかの層に対応している．

- アプリケーション層

TCP/IP 階層の最上位に位置し，OSI 階層モデルのアプリケーション層，プレゼンテーション層，セッション層に対応する．TCP/IP を使用してデータを配送するアプリケーションプログラムプロトコルはこの層に含まれ，そのほとんどはユーザサービスを提供する．具体的には TELNET, FTP, SMTP, DNS, HTTP などのプロトコルがこの層に含まれる．

- トランスポート層

OSI 階層モデルのトランスポート層に対応し，異なるホスト上に存在するアプリケーションのプロセス間通信を行う．この層のプロトコルには TCP, UDP が含まれる．TCP は通信に先立って仮想的な通信路を設定してから通信を行うコネクション型通信で，通信の信頼性が高く要求されるサービスに適している．また，UDP はコネクションレス型通信で，信頼性が低い代わりに簡単にサービスを実現することができる．

- インターネット層

OSI 階層モデルのネットワーク層に対応し，ホストとホストの間の通信を行う．この層には IP プロトコルが含まれる．IP データグラムと呼ばれる情報の基本単位をインターネットを通じて転送，ルーティングし，下位層のネットワークインターフェイス層に渡す．

- ネットワークインタフェース層

OSI 階層モデルのデータリンク層と物理層に対応し，ケーブルを通して最終的な目的地であるパソコンやワークステーションにデータグラムと呼ばれる情報の基本単位（パケットの一種）を送信する．データグラムサービスでは，各パケット（データグラム）はそれぞれ独立して送信されるので，送信したパケットの順序と受信側のパケットの順序が必ずしも一致しない場合がある．この場合には上位層で順序制御を行う．イーサネット，FDDI などがこれに含まれる．

TCP/IP による通信 TCP/IP を使ってデータがネットワークへ送信される時には階層の上から下へ，ネットワークから受け取られるときには下から上へと渡される（図3）．

データがアプリケーション層からネットワークへ送信されるまでには，データの受け渡しを確実にを行うための制御情報がそれぞれの階層でデータの先頭部分にヘッダとして付加される．

UDP UDP はトランスポート層のプロトコルで，アプリケーションからアプリケーションへの通信を提供する．IP がインターネットアドレスをもとにパケットの交換を行うのに対し，UDP はポートという概念でパケットの交換を行う．IP で確立された通信路上にポートという出入り口を持った通信路が仮想的に複数確立される．

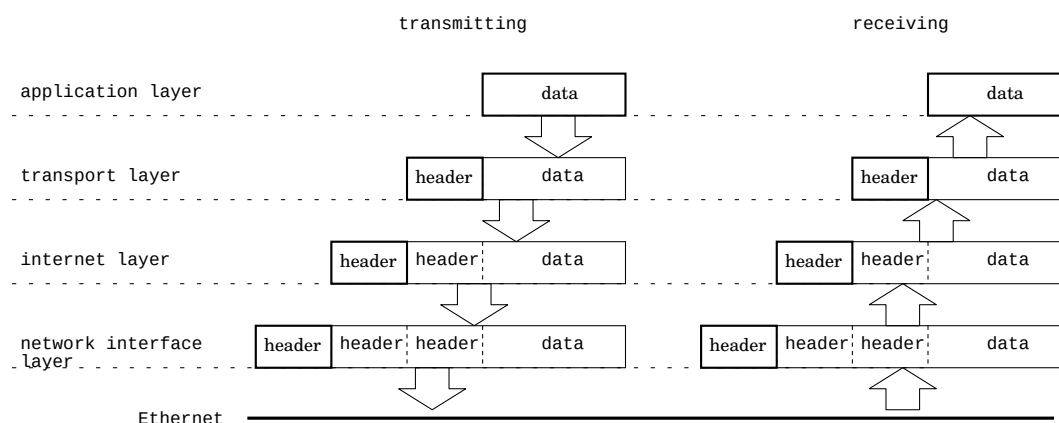


図 3: TCP/IP data transportation

UDP は TCP のコネクション型通信と比べ、信頼性の低いコネクションレス型の通信を行い、誤り制御、順序付け、フロー制御を行わない。そのため、UDP モジュールの処理が単純になり、TCP に比べ高速なデータ転送が可能になる。このように IP がもつ最低限の機能のみをユーザアプリケーションに提供しているため、誤り制御などの信頼性については上位プロトコルで対処する必要がある。

IP アドレス ネットワークに接続されたパソコンやワークステーションで通信を行うためには、それぞれのコンピュータを区別するためのアドレスが必要である。インターネット上の（正確にはインターネットプロトコルで通信を行う）装置に割り当てられるアドレスをインターネットアドレス（IP アドレス）と言い、“172.24.4.1”のようにドットで区切られた4つの10進数で表される。

インターネットに接続されるすべての装置にはユニークな IP アドレスが割り当てられるようになっており、それらはすべて NIC（Network Information Center）で管理されている。そのなかでも、日本で使用される IP アドレスは JPNIC で管理されている。

しかし、インターネットが急速に普及するにつれて IP アドレスの不足が心配されるようになってきた。そのため、組織内部だけのクローズな LAN 環境では、その組織だけで通用する IP アドレス（プライベートアドレスと呼ぶ）を利用し、インターネットにアクセスする場合だけ本来のユニークなアドレス（グローバルアドレス）を割り当てる方法が利用されるようになった。

プライベートアドレス空間からグローバルアドレス空間（インターネット）をアクセスできるようにするしくみとしては、プロキシや NAT（Network Address Translator）が利用される。

愛知県立大学学内ネットワークではプライベートアドレスを使用しており、この IP アドレスは学内でのみ利用可能である。学内のコンピュータからグローバルアドレス空間へのアクセスは、HTTP プロキシおよびファイアウォールの NAT により自動的にアドレス変換を行ってからインターネットへ接続される。

TCP/IP についてさらに勉強したい人には、文献 [4] がわかりやすい。

2.2 ネットワークプログラミング

2.2.1 クライアントサーバモデル

クライアントサーバモデルはネットワークアプリケーションの形態の一つで、一つのアプリケーションがクライアント（サービスを要求するもの）とサーバ（サービスを提供するもの）で構築されている。クライアントとサーバはネットワーク上でお互いに通信を行いながら具体的な処理を行う。

一般的にサーバはクライアントからの要求を待ち、要求に対応した処理を行い、その結果をクライアントへ送信する。たとえば、WWW はクライアントサーバモデルのシステムであり、Firefox, Internet Explorer などのクライアントと、Apache などのサーバが HTTP による通信を行っている。

本実験では、クライアントとサーバの二つのプログラムがネットワーク上でお互いに通信を行いながら具体的な処理を行うプログラムを作成する。

2.2.2 ソケットインタフェース

ソケットインタフェースは、トランスポート層とアプリケーション層の間のインタフェースで、ユーザが作成するアプリケーション層のプログラムから、トランスポート層のプロトコルモジュール（TCP, UDP）を呼び出すために使用される。そのため TCP/IP プロトコルでは、トランスポートプロトコルとして TCP を使うか UDP を使うかを指定することができる。

ここでは、UDP によるコネクションレス型通信の基本的手順を説明する。

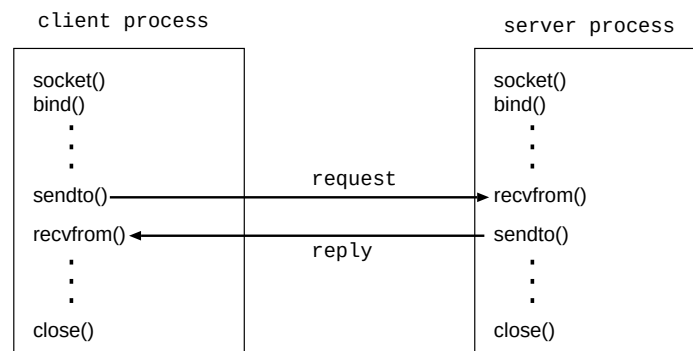


図 4: Connectionless style communication

コネクションレス型通信プログラムで、ソケットインタフェースのシステムコールを使用する例を図 4 に示す。プログラム内で呼ばれるシステムコールを説明する。

1. `socket()`

クライアント、サーバともに、通信の最初に `socket()` を呼び出す。これにより、通信を行うための出入り口となる通信ポートを作成するとともに、そのポートで使用するプロトコルを定める。

2. `bind()`

作成したソケットを識別するためのアドレスを与える。

3. `sendto()`, `recvfrom()`

通信相手のソケットアドレスを指定して、メッセージを送ったり受け取ったりする。クライアントは `sendto()` でサーバへ要求を送り、その後 `recvfrom()` でサーバからの返答を受け取る。サーバは `recvfrom()` でクライアントからの要求を受取り、処理を行った後、クライアントへ `sendto()` で返答する。

4. `close()`

データ転送終了時にソケットをクローズする終了処理を行う。close 実行後は、ソケットが解放されるためデータ転送することができない。

各システムコールの詳細は付録を参照のこと。

2.2.3 ポート番号

ポート番号にはユーザが自由に使えるものと、すでに決まったサービスのために割り当てられているものがある。ワークステーションがアプリケーションで使用する割り当て済みのポート番号は、`/etc/services` ファイルに記述してある。

表 1: ポート番号の割り当て

ポート番号	用途
0 – 255	ウェルノウンポート (telnet, ftp などの良く使われるサービス用)
256 – 1023	予約済ポート (スーパーユーザプロセスのみ使用可能)
1024 – 5000	自動割り当てポート (ポートを指定しなかった場合に自動的にシステムが割り当てる)
5001 – 65535	ユーザが自由に使えるポート

2.3 音声通信

本節では、3 日目の音声通信実験に関わる解説を示す。

2.3.1 音声データ

音声はもともと空気の振動であり、直接コンピュータで扱うことはできない。そこでマイクを用いることにより空気の振動を電圧の振動（アナログ電気信号）に変換することができる。カセットテープ、アナログ電話、ラジオなどが音声のアナログ電気信号を用いた例である。さらに、A/D 変換回路を用いることによって音声のアナログ信号をデジタル信号に変換することができる。デジタル信号化された音声は、データとしてコンピュータで扱うことが可能となる。音声データはデジタル化に伴うサンプリング周波数や量子化ビット数、データの圧縮方法の違いにより多くのデータフォーマットが存在する。代表的な音声データフォーマットには、MP3, WAV, AU などがある。

2.3.2 Linux で扱う音声データ

Linux OS では、`/dev/dsp` で表される音声デバイスから音声を音声データとして入力し、同じ `/dev/dsp` に音声データを出力することで音声を出力することができる。音声データの入出力は音声デバイスに対しそれぞれシステムコールの `read` と `write` を使用する。

2.3.3 音声データと通信

音声データはマルチメディアデータではあるが、通信を行う上では単なるデータとして扱えばよいので、文字データの場合と同様に通信を行うことができる。1 日目と 2 日目には、文字通信実験を行うが、それと同じ方法で 3 日目の音声データ通信も行うことができる。

3 実験

本実験は、複数名で構成されるグループで実施する。全体を通じ、多くのプログラミングを行う必要があるが、基本的に、まずは 1 人 1 人がプログラミングを行うこと（これは予習時にも可能）。プログラミングができれば、作成した通信プログラムを実行することになる。

通信プログラムを実行するときには、1 グループで 2 台以上のワークステーションを使用し、1 台をサーバ、その他をクライアントとしてそれぞれサーバプログラムとクライアントプログラムを実行する。実験においては、各自の作成したプログラムを実行するため、同じような実験をグループの人数分行うこととなる。実験レポートには、自分の作成したプログラムとその実行結果を記載すること。独力でのプログラム作成に時間がかかり完成しなかった場合には、グループ内の他の人のプログラムを参考にしても良いが、どこまでを独力で行い、どこから他の人のプログラムを参考にしたのかをレポートに明記すること（参考元のメンバー名も記載）。

なお、他の人のプログラムの実行結果を書くのはオプションとするが（自分のプログラムの実行結果を書くのは必須）、もし書き加える場合は、プログラム作成者名を明記した上で実行結果を記載すること。その際、もしプログラムの内容自体が自分のものと異なる場合は、他の人のプログラムの中で自分のプログラムと異なる部分を抜き出して記載すること。また、結果を書いたものについては考察も記すこと。

3.1 1日目：文字通信実験1（クライアントプログラムの作成）

通信で送られてきた ASCII 文字（半角英数文字、記号などの 128 文字）を大文字化し、クライアントへ返信するサーバプログラム（実行ファイル）が用意してある。このサーバに文字列を送信し、その結果返ってきた文字列を画面上に表示するクライアントプログラムを次の条件にしたがって作成せよ。穴空きプログラム（sample1.c）が/cis/public/jikken/jikken1/C-1 に置いてあるので、これをコピーしてファイル名を変更し、プログラムを完成させること。

作成するプログラムでは適切なエラー処理を行い、入力した引数がエラーの時には Usage（使用方法）を出力すること。その他、3.4 節の注意を良く読んで実験を行うこと。

作成した通信プログラムを実行する場合には、サーバプログラムとクライアントプログラムを実行する必要があるが、クライアントプログラムは作成したものを実行し、サーバプログラムは/cis/public/jikken/jikken1/C-1/sv を実行すること。

1. クライアントプログラムの条件

- クライアントプログラムは次のように実行することとし、実行時の引数でサーバホスト名を指定する。

client_% プログラム名 サーバホスト名

ホスト名から IP アドレスへの変換は getaddrinfo ライブラリ関数（付録）を使用する。

- 2.2.2 で紹介した通信手順に則ること。

- 通信プロトコル

－ 一度に送信するバッファのサイズ：MAXLINE = 512

－ ポート番号：SERV_UDP_PORT

（プログラム内で適切なポート番号を指定すること）

- コンパイル時に使用するライブラリ

ns1

2. サーバプログラムの実行方法

- サーバプログラム /cis/public/jikken/jikken1/C-1/sv の実行時には、次のように使用するポート番号を引数で与えること。

server_% /cis/public/jikken/jikken1/C-1/sv ポート番号

- サーバプログラムを先に実行してからクライアントプログラムを実行すること。

3.2 2日目：文字通信実験2（サーバプログラムの作成）

2日目はサーバプログラムを作成し，1日目に作成したクライアントプログラムと合わせて実行する。

1日目に使ったサーバプログラムのソース（sample2.c）が，
/cis/public/jikken/jikken1/C-1 に置いてある．これをコピーし，ファイル名を変更する．まず予備実験として，そのファイルをコンパイルして実行し動作することを確認する．

次に，サーバプログラムを書き換え，自分の好きな機能が実行できるものを作成せよ．時間があれば，複数種類のサーバプログラムを作成しても良い．サーバプログラム作成後は，1日目同様に，プログラムの機能を実験により確認すること．クライアントプログラムは，1日目に作成したものを用いること．

レポートには，作成したサーバプログラムを付けプログラムの説明を記すことはもちろんだが，まず最初に，その機能について説明すること．

サーバプログラムの機能の例（自分独自のものならなお良い）：

- 小文字は大文字に，大文字は小文字に変える
- 数字が入っていたら削除する
- 数字が入っていたら空白文字に変える
- 文章校正機能（例えば，先頭の文字は大文字にし，ピリオドの次の文字があれば，大文字に変えた上でピリオドの直後に空白文字を入れる）
- 暗号化/復号機能（例えば，文字を逆順に並び替える）
- 頻度分析（例えば，各文字の出現回数を返す）
- 符号化機能（例えば，2桁の16進数で各文字のASCIIコードを返す）
- 誤り訂正符号化機能（例えば，各文字を3回繰り返す）
- 特定文字のコピー作成機能（例えば，文字列中に“A”が入っていたら“AAA”に変換する）
- 隠し文字検出機能（例えば，文字列中に“L”，“O”，“V”，“E”が全て入っていたら“suki”を返し，そうでなければ，“once again”を返す）
- 占い機能（例えば，文字列中の“S”，“A”，“B”，“C”，“D”の各文字の頻度をカウントし，GPAの値を返す）

3.3 3日目：音声通信実験

本実験では、マルチメディアデータである音声データを対象に、1, 2日目の文字通信実験と同様のクライアントサーバモデル通信プログラムを作成する。クライアントプログラムはマイク入力された音声データを文字通信実験と同様にUDPプロトコルを用いてサーバプログラムへ送信し、サーバプログラムは受信した音声データをスピーカを通して音声出力する。サーバプログラムはすでに用意してあるものを用い、そのサーバにデータを送信するクライアントプログラムの作成を行う。

1. 音声入力プログラム（ソースプログラム）

`/cis/public/jikken/jikken1/C-1/parec-simple.c` は、マイクから入力した音声信号を標準出力へ出力するプログラムである。このプログラムをコピーし、ファイル名を変更した後、コンパイル、実行せよ。そして、端末にデータが表示されることを確認せよ。

【補足】コンパイル時には、ライブラリ (`-lpulse -lpulse-simple`) を設定すること。

プログラムの終了はC-c (コントロールキー+c) を入力すること。次に、標準出力されるデータをリダイレクト ">" を使用して、ファイルに保存せよ。保存したデータが `/cis/public/jikken/jikken1/C-1/pacat-simple` プログラムを使って音声出力されることを確認せよ。

注意事項 プログラム実行時に音声デバイス（マイク入力、スピーカ出力）のボリュームが低すぎると動作確認することができない。必要に応じてデスクトップメニューの [システム], [設定] から [音量調節ツール] を起動し、マイク入力ボリュームとスピーカ出力ボリュームを調節せよ。

2. 音声入力プログラム `parec-simple.c` を理解し、`main` 関数のフローチャートを作成せよ。
3. 文字通信実験で作成した通信プログラムと音声入力プログラム `parec-simple.c` を参考に、音声入力クライアントプログラムを作成せよ。このとき、作成したクライアントプログラムから送信された音声信号が、サーバプログラムで正しく再生されること。

(a) クライアントプログラムの条件

- 実行時の引数でサーバホスト名を指定すること。
`client_% プログラム名 サーバホスト名`
- 解説で紹介したソケットインタフェースのUDP通信手順に則ること。
ただし、今回のサーバプログラムは、データを受信した後に応答を返さない。そのため、クライアントプログラムはサーバからの応答を待つ (`recvfrom` する) 必要がない。
- 一度に送信するバッファのサイズ: `MAXLINE = 4096`

(b) サーバプログラムの実行方法

- 本実験用のサーバプログラムは `/cis/public/jikken/jikken1/C-1/psv` なのでこれを実行すること。実行時には次のように使用するポートの番号を引数で与えること。
`server_% /cis/public/jikken/jikken1/C-1/psv ポート番号`
また、実行時に `-v` オプションを指定すると、サーバが受信したデータを表示することができるので、これを有効に活用すること。
- サーバプログラムを先に実行してからクライアントプログラムを実行すること。

3.4 実験の注意

3.4.1 プログラミングについて

- `make` コマンドを使用してコンパイルすること。(付録参照)
- プログラムのデバッグを行うときに、デバッガを使用すると効率の良いデバッグを行うことができる。(付録参照)
- コンパイルオプション
必要に応じて `-c`, `-o`, `-l`, `-I`, `-L` などのコンパイルオプションを使用すること。
 - `-c` オプション
コンパイルを行った後、実行ファイルを出力せずオブジェクトファイル(`.o` ファイル)を出力する。
 - `-o` オプション
出力するファイル名を指定する。
 - `-l` オプション
実行ファイル作成時にリンクするライブラリを指定する。
 - `-I` オプション
インクルードファイルの置いてあるディレクトリを指定する。`/usr/include` に置いてあるインクルードファイルは、このオプションがなくてもインクルードできる。
 - `-L` オプション
ライブラリの置いてあるディレクトリを指定する。`/usr/lib` に置いてあるライブラリは、このオプションがなくてもリンクすることができる。
- オンラインマニュアル
UNIX コマンド, システムコール, ライブラリ関数の使用方法是オンラインマニュアル(`man` コマンド)を活用して調べること。

例: オプション解析を行うライブラリ関数 `getopt` の使用方法
`% man -s 3c getopt`

4 課題

(1 日目の実験関連)

課題 1-1 本実験では UDP というコネクションレス型の通信プログラムを作成したが、TCP を利用してプログラムを作成する場合、どのような点が異なるか 2.2.2 のソケットインタフェースの手順と比較して具体的に説明せよ。

課題 1-2 1 日目の実験で作成したプログラムを、効率的なプログラム、他人から見て読みやすいプログラムに改良するとしたら、どのような点が考えられるか検討せよ。

課題 1-3 1 日目の実験を行って、難しかった点、興味を持った点などについて感想を述べよ。

(2 日目の実験関連)

課題 2-1 サーバプログラムの機能として、実験で行わなかったものを複数考え、その機能（仕様）を詳しく説明せよ。1 台のサーバと複数台のクライアントが通信する場合の機能なども考えて良い。

課題 2-2 2 日目の実験を行って、難しかった点、興味を持った点などについて感想を述べよ。

(3 日目の実験関連)

課題 3-1 本実験で音声データの通信を行ったが、計算機で音声データを扱うプログラムを作成する場合、通信を行うプログラムを作成する場合、それぞれ注意すべき点を検討せよ。

課題 3-2 3 日目の実験を行って、難しかった点、興味を持った点などについて感想を述べよ。

(全体として)

課題 4-1 通信プログラムの応用として、これまでにない新しいアプリケーションを考えよ。また、そのアプリケーションの実現に必要な技術を検討せよ。

課題 4-2 通信プログラムは便利に使うことができる一方、ネットワークセキュリティに関する問題と切り離すことができない。現在、自分で行っているネットワークセキュリティ対策と、さらに行う必要があると考えるセキュリティ対策を検討せよ。

参考文献

- [1] Douglas Comer (著), 村井純, 楠本博之 (訳): “TCP/IP によるネットワーク構築 vol.I — 原理・プロトコル・アーキテクチャ —”, 共立出版, (1993).
- [2] Douglas Comer, David Stevens (著), 村井純, 楠本博之 (訳): “TCP/IP によるネットワーク構築 vol.II — 設計・実装・内部構造 —”, 共立出版 v(1995).
- [3] Douglas Comer, David Stevens (著), 村井純, 楠本博之 (訳): “TCP/IP によるネットワーク構築 vol.III — クライアント-サーバプログラミングとアプリケーション —”, 共立出版, (1996).
- [4] 野口正一 (監修), 西田竹志 (著): “図解 TCP/IP 入門”, オーム社, (1993).
- [5] 羽山博 (監修), 金内典充, 今安正和 (著): “プログラミングリファレンス UNIX ネットワークプログラミング”, オーム社, (1993).
- [6] Kevin Washburn, Jim Evans (著), 海江田一詩 (訳): “TCP/IP バイブル改訂新版”, アスキー出版局, (1999).
- [7] “A standard for the Transmission of IP Datagrams on Avian Carriers”, RFC1149, (1990).
- [8] “IP over Avian Carriers with Quality of Service”, RFC2549, (1999).
- [9] 小山裕司, 斎藤靖, 佐々木浩, 中込知之 (著): “UNIX 入門”, トッパン, (1996).
- [10] 山口和紀 (監修): “The UNIX Super Text[下]”, 技術評論社, (1992).
- [11] 棕田實 (著): “改訂第3版 ANSI C 対応 はじめての C”, 技術評論社, (1993).

付録

A.1 システムコール，ライブラリ関数の詳細

文献 [3] より抜粋.

socket

- 形式
`int socket(family, type, protocol);`
- 解説
`socket` はネットワーク通信で使われるソケットを生成し，ソケットに対する識別子を `int` 型で返す.
- 引数

引数	型	意味
<code>family</code>	<code>int</code>	プロトコルまたはアドレスファミリ (TCP/IP に対しては <code>PF_INET</code> , <code>AF_INET</code> を指定する. 本実験では <code>AF_INET</code> を使用する.)
<code>type</code>	<code>int</code>	サービス型 (TCP に対しては <code>SOCK_STREAM</code> , UDP に対しては <code>SOCK_DGRAM</code>)
<code>protocol</code>	<code>int</code>	使用するプロトコル番号. デフォルト値を使用する場合は 0.

- 戻り値
`socket` の呼び出しは成功すると生成したソケットのソケット識別子を返し，エラーが起これると `-1` を返す. エラーが起これったときは，グローバル変数 `errno` にエラーコードが代入される.

bind

- 形式
`int bind(socket, localaddr, addrlen);`
- 解説
`bind` はソケットに対して，ローカル IP アドレスとプロトコルポート番号を指定する. TCP/IP プロトコルの場合には，IP アドレスとプロトコルポート番号の指定に，その両方が含まれる `sockaddr_in` 構造体を用いる. つまり次のように，変数 `addr` が `sockaddr_in` 型で宣言されている場合，`bind` 関数の呼

び出しでは `sockaddr` 型にキャストされて引数に与えられる.

```

    struct sockaddr_in  addr;
        :
    bind(socket, (struct sockaddr *) &addr, addrlen);

```

`sockaddr_in` 構造体は図 5 のように宣言されている.

```

struct in_addr {
    u_long      s_addr;          /* IP address          */
};

struct sockaddr_in {
    short        sin_family;      /* type of address     */
    u_short      sin_port;       /* protocol port number */
    struct in_addr sin_addr;     /* IP address          */
    char         sin_zero[8];    /* unused (set to zero) */
};

```

図 5: definition of `struct sockaddr_in` in `/usr/include/netinet/in.h`

- 引数

引数	型	意味
<code>socket</code>	<code>int</code>	<code>socket</code> 呼び出しでつくられたソケット識別子
<code>localaddr</code>	<code>&sockaddr</code>	IP アドレスとプロトコルポート番号を指定する アドレス構造体
<code>addrlen</code>	<code>int</code>	アドレス構造体の大きさのバイト数

- 戻り値

`bind` の呼び出しは成功すると 0 を返し, エラーが起こると -1 を返す. エラーが起こったときは, グローバル変数 `errno` にエラーコードが代入される.

sendto

- 形式

```
int sendto(socket, msg, msglen, flags, to, tolen);
```

- 解説

`sendto` は, アドレス構造体から終点アドレスを得てそこへメッセージを送る.

- 引数

引数	型	意味
socket	int	socket 呼び出しでつくられたソケット識別子
msg	&char	メッセージへのポインタ
msglen	int	メッセージのバイト数
flags	int	帯域外データもしくはメッセージ先読みを指定する制御ビット (本実験では特に使用しないので引数に 0 を与える)
to	&sockaddr	アドレス構造体へのポインタ
tolen	int	アドレスのバイト数

- 戻り値

sendto は成功すると送ったバイト数を返し、エラーが起こると -1 を返す。エラーが起こったときは、グローバル変数 errno にエラーコードが代入される。

recvfrom

- 形式

```
int recvfrom(socket, buffer, buflen, flags, from, fromlen);
```

- 解説

recvfrom は、ソケットに到着した次のメッセージを取り出し、送信者のアドレスを記録する。

- 引数

引数	型	意味
socket	int	socket 呼び出しでつくられたソケット識別子
buffer	&char	メッセージを保持するバッファのアドレス
buflen	int	バッファのバイト数
flags	int	帯域外データもしくはメッセージ先読みを指定する制御ビット (本実験では特に使用しないので引数に 0 を与える)
from	&sockaddr	送信者のアドレスを保持する構造体のアドレス
fromlen	&int	from バッファの長さ、送信者のアドレスの大きさを返す

- 戻り値

recvfrom は成功すると受け取ったメッセージのバイト数を返し、エラーが起こると -1 を返す。エラーが起こったときは、グローバル変数 errno にエラーコードが代入される。

close

- 形式

```
int close(socket);
```

- 解説

アプリケーションでソケットの使用が終了したときには、close を呼び出す。close は通信を終了させ、ソケットを解放する。

- 引数

引数	型	意味
socket	int	クローズされるソケットの識別子

- 戻り値

close の呼び出しは成功すると 0 を返し、エラーが起こると -1 を返す。エラーが起こったときは、グローバル変数 errno にエラーコードが代入される。

getaddrinfo ライブラリ関数

- 形式

```
int getaddrinfo(const char *node, const char *service, const struct  
addrinfo *hints, struct addrinfo **res);
```

- 解説

getaddrinfo() は、(インターネットのホストとサービスを識別する) node と service を渡すと、一つ以上の addrinfo 構造体を返す。それぞれの addrinfo 構造体には、bind(2) や connect(2) を呼び出す際に指定できるインターネットアドレスが格納されている。

- 引数

引数	意味
node	ホスト名を指定する
service	サービスを指定する。本実験では NULL を指定する。
hints	IP アドレス情報のためのヒントを与える。
res	IP アドレス情報が格納される。

本実験で hints には以下の情報を指定する。

- hints.ai_socktype = SOCK_DGRAM;
- hints.ai_family = AF_INET;

- 戻り値

`getaddrinfo()` は成功すると 0 を返し、失敗すると 0 以外のエラーコードを返す。エラーコードは省く。

A.2 make コマンド

`make` コマンドは、カレントディレクトリにある `Makefile` という名前のファイルを読み込み、そこに書いてあるコンパイル手順に従ってコンパイルを行うコマンドである。図 6 は `Makefile` の一例である。

```
# Example Makefile

program: program.o
    cc -o program program.o -lnsl -m32

program.o: program.c
    cc -c program.c

clean:
    rm -f program *.o *~
```

図 6: `Makefile` の例

この `Makefile` のおいてあるディレクトリで `make` を実行すると次のように表示され、実行ファイル `program` が作成される。

```
% make
cc -c program.c
cc -o program program.o -lnsl -m32
%
```

`Makefile` の規則

`Makefile` は、図 7 の形をした規則の集まりで書かれており、「目的ファイル」は作成する目的のファイル、「依存ファイル 1 …」は「目的ファイル」を作成するために必要なファイル、「実行コマンド」は「目的ファイル」を作成するためのコマンドを表している。

書き方の注意として、

- 目的ファイルの後に “:” を書くこと
- 実行コマンド行の先頭には（空白ではなく）タブを入れなければならない

● 行の始めに “#” を入れるとその行はコメントになる
などがある。

目的ファイル: 依存ファイル 1 依存ファイル 2 … 実行コマンド

図 7: Makefile の規則

図 6 の Makefile で `make` を実行すると、最初に出てくる目的ファイル “program” を作成しようとし、依存ファイル “program.o” がカレントディレクトリに存在するか探す。program.o がいない場合、program.o を目的ファイルとする規則を使用して program.o を作成する。よって、

```
cc -c program.c
```

が実行され、次に program を作成する規則により、

```
cc -o program program.o -lnsl -m32
```

が実行される。

また、make コマンドは目的ファイルを引数で指定することができる。図 6 の Makefile で `make clean` を実行すると、目的ファイル “clean” を作成するための規則から実行コマンドが実行され、program、オブジェクトファイル *.o、mule のバックアップファイル*~ が消去される。ただしこのとき、clean というファイルは実際には存在しない。

タイムスタンプ

make コマンドを 2 回連続で実行しても、1 度しかコンパイルされない。これは make コマンドが目的ファイルとその目的ファイルを作るのに必要な依存ファイルのタイムスタンプ(ファイル作成時刻)を調べており、その結果、目的ファイルが全ての依存ファイルよりも新しければ、make は何も実行せずに終了してしまうからである。

一方、依存ファイルを修正した場合、タイムスタンプが変更されるので、その後に make を実行すると Makefile 内の規則のうち、修正したファイルに依存関係のある部分だけコマンドが実行される。

マクロ

Makefile はマクロを使って文字列を置き換えることにより、簡潔に書くことができる。図 6 の Makefile はマクロを使用すると図 8 のように書くことができる。このようにマクロを使ってファイル名、ライブラリ名を定義しておけば、ファイル名や使用するライブラリを変更しても Makefile の修正を簡単に行うことができる。

make コマンドの詳細については、`man make` もしくは文献 [9] の 10.1 章を参考にするとよい。

```

# Example Makefile

CC = cc
LIB = -lnsl
PROG = program
RM = rm -f

$(PROG): $(PROG).o
        $(CC) -o $(PROG) $(PROG).o $(LIB)

$(PROG).o: $(PROG).c
        $(CC) -c $(PROG).c

clean:
        $(RM) $(PROG) *.o *~

```

図 8: マクロを使った Makefile の例

A.3 デバッガ (gdb) の使い方

コンパイルはできたが、出力された実行ファイルを実行すると Segmentation fault, Bus error などのエラーが発生してしまう場合、デバッグツールの dbx や gdb を使用してデバッグすること。以下に gdb の使い方の概要を示すので、詳細は文献 [9] 10.1 章を参照すること。

1. 次のソースプログラム hoge.c を例に gdb の使い方を説明する。

```

int a[100];
hoge(int x){
    printf("a[%d]=%d\n", x, a[x]);
    hoge(x*10);
}
main(){
    hoge(10);
}

```

2. 準備

デバッガ gdb を使用するための準備として、コンパイル時に -g オプションを付加してコンパイルすること。コンパイル (.c から .o) とリンク (.o から 実行ファイル) をわけて行っている場合、リンクではなくコンパイル時に -g オプションを使用すること。

```
% cc -g hoge.c
```

3. gdb の起動

% gdb 実行ファイル名

で起動する。すると次のような gdb のプロンプトが出て、デバッグモードに入る。

```
% gdb a.out
GNU gdb (GDB) Red Hat Enterprise Linux (7.0.1-23.el5_5.2)
Copyright (C) 2009 Free Software Foundation, Inc.
...
Reading symbols from ...
(gdb)
```

4. 作成したプログラムの実行

(gdb) run 引数

このように実行ファイル名のかわりに run で実行する。すると、通常実行時と同様にエラーが発生しプログラムは終了するが、エラーの場所などが表示される。

```
(gdb) run
Starting program: a.out
a[10]=0
a[100]=0

Program received signal SIGSEGV, Segmentation fault.
0x080483ad in hoge (x=1000) at hoge.c:3
3      printf("a[%d]=%d\n", x, a[x]);
(gdb)
```

5. プログラムの状態を調べるコマンド

以下のコマンドを使用して、プログラムが停止したときの状態を調べることができる。

- where

関数の呼び出し関係を下から表示する。この場合、main 関数の中で関数 hoge が呼び出され、関数 hoge が 3 回入れ子になって呼び出された状態でエラーになったことがわかる。

```
(gdb) where
#0  0x080483ad in hoge (x=1000) at hoge.c:3
#1  0x080483df in hoge (x=100) at hoge.c:4
#2  0x080483df in hoge (x=10) at hoge.c:4
#3  0x080483fe in main () at hoge.c:7
(gdb)
```

- print

変数の値を表示する。配列、構造体なども表示できる。

```
(gdb) print x
$1 = 1000
(gdb) print a[1000]
Cannot access memory at address 0x804a620
(gdb)
```

- up
関数の呼び出し関係の一つさかのぼる。逆に down では呼び出された方へ一つ進む。

```
(gdb) up
#1 0x080483df in hoge (x=100) at hoge.c:4
4      hoge(x*10);
(gdb) where
#0 0x080483ad in hoge (x=1000) at hoge.c:3
#1 0x080483df in hoge (x=100) at hoge.c:4
#2 0x080483df in hoge (x=10) at hoge.c:4
#3 0x080483fe in main () at hoge.c:7
(gdb) print x
$2 = 100
(gdb)
```

6. 他にも, next(一行実行. ただし関数呼び出しを一行と見なす), step(一行実行. 呼び出された関数の中も一行ずつ実行する), help(ヘルプの表示), quit(デバッグの終了) や, ブレークポイントの設定, 変数値の設定, 関数の実行などの命令があるので, help 命令や文献を参考にとすると良い.

情報科学実験Ⅰ 通信ネットワーク

ネットワークシステムの性能評価と設計¹

田 学軍²

¹2022 年度応用実験の資料 (奥田、田) をベースに作成された

²居室：E413, 内線 2413, E-mail：tan@ist.aichi-pu.ac.jp

目次

第Ⅰ部 第1日目前半	4
1 実験の背景と目的：離散シミュレーションと待ち行列理論	4
2 実験の背景と目的：情報システムとの関連	5
3 実験スケジュール	6
4 注意事項	7
4.1 レポートについて	7
第Ⅱ部 第1日目後半	8
5 情報ネットワークシステムの設計：モデリングと性能評価	8
5.1 待ち行列理論と基本公式	8
5.2 M/M/1	9
5.3 M/M/s	9
5.4 M/M/1/K と M/M/s/K	10
6 S4 演習：銀行の窓口モデル (M/M/1/K と M/M/1/∞)	10
6.1 プロジェクトを作成する	11
6.2 シミュレーションモデルを作成する	12
6.3 モデルパラメータの設定とシミュレーションの実行	17
6.4 様々な検証	20
6.4.1 ベテラン行員	20
6.4.2 行員数を増やす	20
第Ⅲ部 第2日目	22
7 M/M/s と M/M/s/K	22
7.1 評価モデル	22
7.2 M/M/1/K	23
7.2.1 M/M/1/K モデルの作成	23
7.2.2 パラメータの設定	23
7.2.3 実行結果の表示	29
7.2.4 廃棄パケットの記録	29
7.3 M/M/S/K	33
7.3.1 M/M/S/K モデルの作成	33
7.3.2 実行結果の表示及び理論値との比較	35
7.4 実例を用いたシミュレーション	35
7.4.1 実例シミュレーションに関するパラメータ換算	35
7.4.2 M/M/1/K 以外の例のシミュレーション	36

第Ⅳ部 第3日目	37
8 形態分析	37

第I部

第1日目前半

1 実験の背景と目的：離散シミュレーションと待ち行列理論

君たちは様々なサービスを利用している．インターネットを活用したサービス，スーパーマーケットやファストフードなども含まれる．

今，ある店舗が開店したことを想像して欲しい．店主はそれほど客が来店しないことを想定していたため，レジのアルバイトを1名だけ雇用していた．その結果，図1のような行列が出来てしまった．このような状況を待ち行列理論 [9, 11] では，ケンドールの記号で表記する．ここでは客はランダムに到着し (M)，レジの時間もランダム (M)，待機スペースは無限大だと仮定するので， $M/M/1/\infty$ （あるいは無限記号を省略した $M/M/1$ ）と表記する．待機スペースが有限整数 K の場合は $M/M/1/K$ と表記する．



図 1: 一つの普通の M/M/1

そこで店主は翌日からレジの改善をすることにした．その際，図2，図3，図4に示すような方式である．図2はレジを並列に増やし，そのレジに対して行列を作る方式である．客は好きなレジを選択することができる．一方，図3は行列を一行にして，空いたレジに客が移動する方式である．この方式では客はレジを選ぶことが出来ない．なお図4は，レジは一人のままとして，ベテランで処理時間がとても短い優秀な人を雇用しているのである．

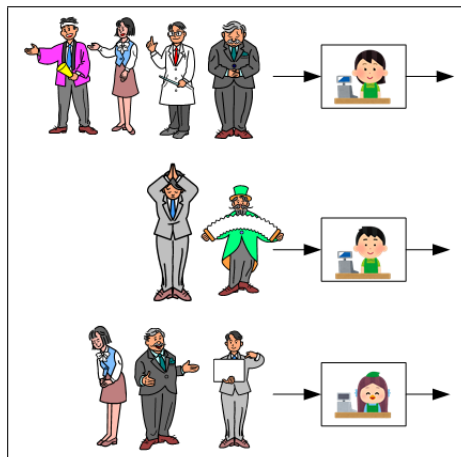


図 2: 複数の M/M/1

さてどの方式が客にとっても，店舗にとっても良いのだろうか．このことを考えるのが，「待ち行列の形態分析」と言われている問題である．実際の店舗で実験するのも良いだろうが，実際の店舗での実験にはコスト（時間，人件費（レジ係役，店員役））がかかる．そこで利用する技術（方



図 3: 一つの M/M/s

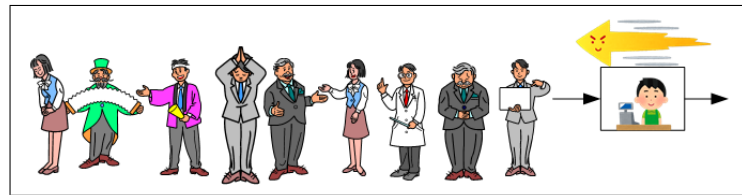


図 4: 一つの超高速な M/M/1

法論) が, 本実験で実習する離散シミュレーションであり, 学部3年生前期で学ぶ待ち行列理論である. なお離散シミュレーションとは確率的に発生する事象を模擬する方法である. 上記の例では客の到着間隔, 客のレジでのサービス時間が確率的に変動すると仮定している.

ところで今回は客とレジ係を用いて現象を説明したが, 抽象化して考えておくと, 様々な領域において離散シミュレーションや待ち行列理論を活用することが出来る. 図5, 図6, 図7によく利用される抽象化モデルをまとめて示す.

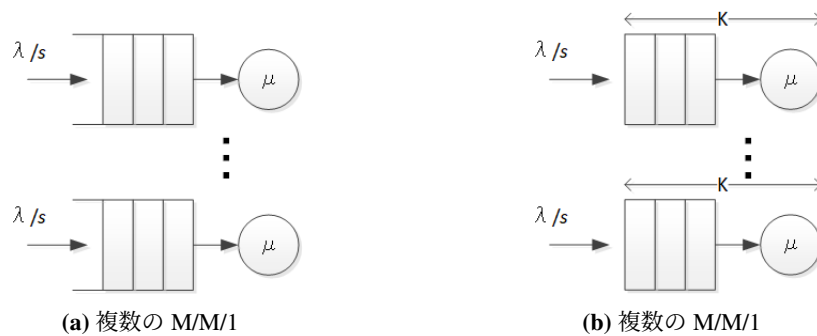


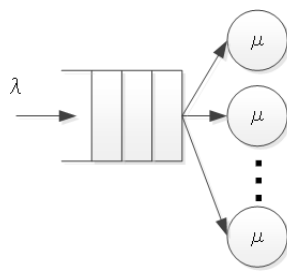
図 5: 複数の M/M/1

2 実験の背景と目的：情報システムとの関連

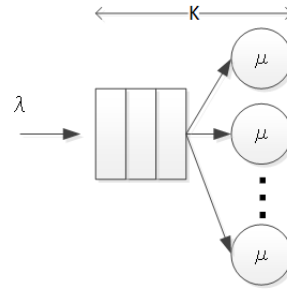
情報システム (information system) とは, キーマンズネットワーク (<http://www.keyman.or.jp>)³の運営する「IT 単語帳」には次のように説明されている.

情報を適切に保存・管理・流通するための仕組み. 通常, コンピュータとネットワーク, 及びそれを制御するソフトウェア, その運用体制までを含んだものを指す. コン

³ (株) リクルートが運営する, IT 製品に関する情報を Web・メールにて提供するサービス.



(a) 1 個の M/M/s

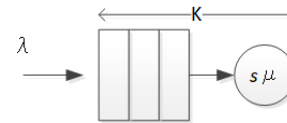


(b) 1 個の M/M/s/K

図 6: 1 個の M/M/s



(a) 一つの超高速 M/M/1



(b) 一つの超高速 M/M/1/K

図 7: 1 個の超高速 M/M/1

ピュータを用いない「情報システム」は，言葉の意味として矛盾しているわけでは
ないが，現代ではほとんどの場合，情報システムは「コンピュータシステム」と同義と
して用いられる．

同様に，(株) インセプト (Incept Inc.) の運営する「IT 用語辞典 e-Words」(<http://e-words.jp/>) には次のように説明されている．

情報を適切に保存・管理・流通するための仕組み．通常，コンピュータとネットワー
ク，およびそれを制御するソフトウェア，その運用体制までを含んだものを指す．コン
ピュータを用いない「情報システム」は，言葉の意味として矛盾しているわけではな
い，現代ではほとんどの場合，情報システムは「コンピュータシステム」と同義と
して用いられる．

つまり，現代の情報システムを学習する上で，コンピュータネットワークや情報通信システムの
原理や設計手法について学ぶことは必要不可欠になっている．この実験ではコンピュータネット
ワークや情報通信システムの設計において不可欠なシミュレーションの基礎を学ぶことになる．

3 実験スケジュール

本実験「ネットワークシステムの性能評価と設計」は，3 日間の実験日の間に，以下の項目につ
いて実験する．

第 1 日目

1. 離散シミュレーションと問題解決
2. M/M/モデルの概説
3. S4 チュートリアル実習

4. M/M/1 モデルの性能評価結果に関する考察

第2日目

1. M/M/s モデルの概説
2. S4 による M/M/s モデルの性能評価
3. M/M/s モデルの性能評価結果に関する考察

第3日目

1. 待ち行列モデルの形態分析の概説
2. S4 による形態分析実習
3. 形態分析結果の考察

4 注意事項

4.1 レポートについて

これらの実験を通して、以下の事を経験し、情報システムの評価と設計に関して理解、興味を深めて欲しい。

1. ネットワークの基礎となる考え方について理解を深める⁴。
2. 離散シミュレーションにより情報システムの設計技法の基礎について学ぶ。

これまでの実験同様、本実験においてもレポートを課す。レポートについての注意事項を下記に示す。

1. レポート提出数：1 通とする。
2. レポート提出日：3 回目の実験終了後一週間後を目処とし、後日指定する。
3. レポート受領方法：提出方法は実験時に説明する。なお受領時に口頭試問を行う場合もある。
4. レポートの作成方法： $\text{\LaTeX}$ により作成すること。なお、理工系の学部における実験レポートのための文章表現については、文献 [1, 2, 3, 4, 5, 6, 7, 8]などを参考にとすると良い。これらは（類書も多くある。自分に合うものを探すと良い）卒業論文や修士論文を書く際にも利用できる書籍である。是非、自分に合う書籍を手元におき、折に触れて参照する習慣を身につけて欲しい。
5. レポート印刷方法：表裏印刷とする。

⁴3 年生や 4 年生向けに開講されている講義「コンピュータネットワーク（3 年前期）」、「通信ネットワーク（3 年後期）」、「オペレーションズ・リサーチ（4 年前期）」などを履修するとさらに理解が深まるであろう。

第II部

第1日目後半

5 情報ネットワークシステムの設計：モデリングと性能評価

コンピュータシステムや通信システム等を効率的に設計するためには，OR(Operations Research)で扱われている待ち行列モデル (Queueing Model) を用いて，システムの性能を評価することが必要である．ここでいう設計とは，「システムの利用形態やパケット発生状況等の条件」の下で，「パケットが処理される時間やパケットが廃棄される確率が，与えられた目標値以下」となるように，「回線の速度，サーバーの数や配置，最適なサーバーの処理能力等」を決定することである [9, 10]．通信システムにおいては待ち行列モデルのことを，通信トラフィックモデル (Teletraffic Model) と表現する．待ち行列理論や通信トラフィック理論については詳しく学習したい人は，文献 [11, 12, 13, 14, 15, 16, 17, 18] 等が参考になるであろう．

通信トラフィックモデルの評価には，

- 解析的手法 (待ち行列理論)
- シミュレーションによる手法

によるものがある．

解析的手法 (待ち行列理論) では，客（この実験ではパケット）の母集団およびサーバー，到着およびサービス時間等の確率法則ならびに行列の規律を規定し，客の滞りや流れの様相を数学的に解析する．詳しくは 5.1 節で述べる．

他方，コンピュータや通信システムの設計では，種々のタイプ，レベル，形態のシミュレーションが行われている．本実験で考察対象とするシミュレーションのタイプは離散イベント・システム・シミュレーションと呼ばれる．離散イベント・システムは状態が離散的（非連続）時間で変化する．例えば，ある PC からのパケットは連続的に発生するのではなく，電子メールの送受信などのイベントにより発生する．離散シミュレーションについて詳しく学びたい人は，文献 [19, 20] が参考になる．

さて，ネットワークの設計において，待ち行列モデルによる応用確率モデルによるアプローチでは限界があるため，モンテカルロシミュレーションを基本とするネットワークシミュレータである Riverbed (旧 OPNET) [21] や ns2[22] が広く利用されるようになった [23]．なお，ユーザが移動するモバイル通信システムの性能評価手法として OPNET と離散イベント・システム・シミュレーションを組み合わせた手法 [24] なども提案されている．さらには計算時間を短縮するためにハイブリッド型シミュレーション手法も提案されている [25]．

本実験では S4(エスクワトロ) Simulation System[26] を利用して実験を行う．S4 は誰でも簡単に複雑なモデルを GUI 上で表現しシミュレーションを行なえる汎用シミュレーションシステムである．

5.1 待ち行列理論と基本公式

待ち行列理論 (Queueing Theory) とは，顧客がサービスを受けるために行列に並ぶような確率的に挙動するシステムの混雑現象を数理モデルを用いて解析することを目的とした理論である．応用数学の一分野である OR (オペレーションズ・リサーチ, Operations Research) の一分野である．

電話交換機や情報ネットワーク、生産システム、空港や病院などの設計や性能評価に応用される。性能評価指標としては、待ち行列長、待ち時間、スループット、呼損率、稼働率、廃棄率等が用いられる。例えば、システムの性能がある設計目標を満たすために必要な設計パラメータを決定する際に利用することができる。本実験においては、ネットワークシステムの平均応答時間がある定められた時間にするために、ネットワークシステムのどのくらいの性能のサーバーを何台おくかということを決定する際に利用することができる。ここで、サーバーとは窓口とも呼ばれ、客とはサーバーのサービスを受けようとする個々の要求のことであり、扱う問題により人であったりパケットあるいは呼といったモノになることもある。

以下には、待ち行列理論の基本的なシステムとして取り上げられる M/M/1 と M/M/s に関する公式を整理しておく。

5.2 M/M/1

M/M/1 は、ポアソン到着で客（実験ではパケット等）が、単一のサーバー（実験ではサーバ PC やプリンタ）へ到着し、そのサービスに要する時間は指数分布に従い、サーバーは先着順にサービスを行うモデルである。なお、バッファは無限にある。すなわち行列に制限はないモデルである。

単位時間内に客がサービスを要求する数の平均を到着率といい λ で、単位時間内にサーバーが完了した平均のサービス数をサービス率といい μ で各々を表現すると、M/M/1 に関する公式が次のように与えられる [9, 11].

$$\rho = (\text{利用率}) = \lambda/\mu \quad (1)$$

$$p_0 = (\text{システム（系）が空である確率}) = 1 - \rho \quad (2)$$

$$p_n = (\text{系内に } n \text{ 人の客がいる確率}) = (1 - \rho)\rho^n \quad (3)$$

$$L = (\text{系平均客数}) = \frac{\rho}{1 - \rho} \quad (4)$$

$$L_q = (\text{列平均客数}) = L - \rho = \frac{\rho^2}{1 - \rho} \quad (5)$$

$$W = (\text{系平均待ち時間}) = \frac{L}{\lambda} = \frac{1}{\mu(1 - \rho)} \quad (6)$$

$$W_q = (\text{列系平均待ち時間}) = \frac{L_q}{\lambda} = \frac{\rho}{\mu(1 - \rho)} \quad (7)$$

$$\Pr\{\text{列待ち時間} > t\} = \rho e^{-(1-\rho)\mu t} \quad (8)$$

5.3 M/M/s

M/M/s は、ポアソン到着、指数サービスで、 s 個の窓口が並列に並んでいるモデルである。客はどのサーバーに行ってもサービスを受けても良いが、全サーバーが塞がっている場合は、1 列の行列を作り、サーバーの一つが空き次第、先着順に空いたサーバに行きサービスを受ける。なお、バッファは無限にある。すなわち行列に制限はないモデルである。

客の到着率を λ 、各サーバーのサービス率は同一で μ とすると、M/M/s に関する公式が次のよ

うに与えられる [9, 11].

$$a = \lambda/\mu \quad (9)$$

$$\rho = (\text{利用率}) = \lambda/s\mu \quad (10)$$

$$p_0 = (\text{システム (系) が空である確率}) = 1/\left\{\sum_{n=0}^{s-1} \frac{a^n}{n!} + \frac{a^s}{(s-1)!(s-a)}\right\} \quad (11)$$

$$p_n = (\text{系内に } n \text{ 人の客がいる確率}) = \begin{cases} a^n p_0 / n! & (0 \leq n \leq s) \\ s^s \rho^n p_0 / s! & (s \leq n) \end{cases} \quad (12)$$

$$L = (\text{系平均客数}) = \frac{s^s \rho^{s+1}}{s!(1-\rho)^2} p_0 + a \quad (13)$$

$$L_q = (\text{列平均客数}) = \frac{s^s \rho^{s+1}}{s!(1-\rho)^2} p_0 = L - a \quad (14)$$

$$W = (\text{系平均待ち時間}) = \frac{s^{s-1} \rho^s}{s! \mu (1-\rho)^2} p_0 + \frac{1}{\mu} = \frac{L}{\lambda} \quad (15)$$

$$W_q = (\text{列系平均待ち時間}) = \frac{s^{s-1} \rho^s}{s! \mu (1-\rho)^2} p_0 = \frac{L_q}{\lambda} \quad (16)$$

$$\Pi = (\text{窓口が全部塞がっている確率}) = \sum_{n=s}^{\infty} p_n = \frac{(\rho s)^s}{s!(1-\rho)} p_0 \quad (17)$$

$$\Pr\{\text{列待ち時間} > t\} = \Pi e^{-(1-\rho)s\mu t} \quad (18)$$

5.4 M/M/1/K と M/M/s/K

M/M/s/K は、ポアソン到着、指数サービスで、 s 個の窓口が並列に並んでいるモデルである。客はどのサーバーに行ってもサービスを受けても良いが、全サーバーが塞がっている場合は、1 列の行列を作り、サーバーの一つが空き次第、先着順に空いたサーバに行きサービスを受ける。なお、バッファは有限サイズであり、システムの中に収容できる客数は K 個である。

6 S4 演習：銀行の窓口モデル (M/M/1/K と M/M/1/∞)

次のように銀行の窓口をモデル化する。

- お客の来店間隔は確率的で平均 60 秒に 1 人来店する（来店間隔は平均 60 の指数分布に従う）
- 銀行には窓口が 1 つある
- 来店したお客は窓口でサービスを受ける
- 窓口にいる銀行の担当者は 1 人である
- 来店したお客は待ち行列に並びサービスを受けるのを待つ
- 先に並んだ人から 1 人ずつ窓口に呼び出されサービスを受ける
- お客が窓口でサービスを受ける時間は確率的で平均 50 秒間サービスを受ける（サービスを受ける時間は平均 50 の指数分布に従う）

- サービスを受けたお客はそのまま退店する
- 行列に並べる人数は 30 人までで、30 人いっぱい並んでいるときは、来店したお客は帰ってしまう

この関係を待ち行列理論におけるケンドールの記号で表記すると、M/M/1/31 となる。このモデルにおいて平均待ち時間、平均行列長などを、実際の観察実験でなく、コンピュータ上のシミュレーション実験で検証することが求められている。

上記のモデルを S4 で実施してみるのが、この演習の目的となる。S4 における大きな流れは次のようになる。

1. プロジェクトを作成する (6.1 節)
2. シミュレーションモデルを作成する (6.2 節)
 - (a) 各アイコンの配置
 - (b) アイコンパラメータの設定
 - i. お客を作成する
 - ii. 来店を作成する-来店間隔
 - iii. 窓口を作成する-窓口の個数
 - iv. 窓口利用を作成する-窓口利用間隔
 - v. 退店を作成する
 - (c) リンクの作成
3. モデルパラメータの設定とシミュレーションの実行 (6.3 節)
 - (a) シミュレーション時間設定
 - (b) 最適化
 - (c) 実験計画
 - (d) モデルを実行する
 - (e) 結果を見る・可視化する
 - (f) 結果サマリ
 - (g) ヒストグラム
 - (h) 窓口のパフォーマンス可視化
4. 様々な検証 (6.4 節)

6.1 プロジェクトを作成する

新規プロジェクトを作成してモデルを構築していく (図 8 参照)。

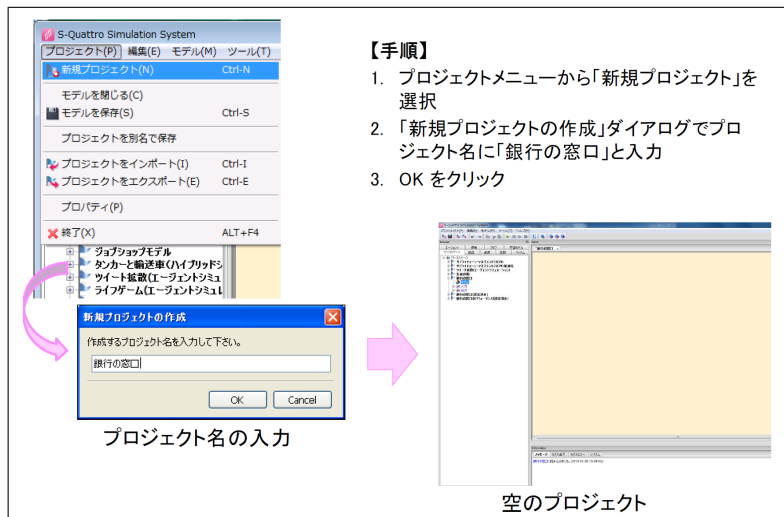


図 8: プロジェクトを作成する

6.2 シミュレーションモデルを作成する

ここではシミュレーションモデルを構築する．具体的には上述した次のことを画面上で実施する．

1. 各アイコンの配置
2. アイコンパラメータの設定
 - (a) お客を作成する
 - (b) 来店を作成する-来店間隔
 - (c) 窓口を作成する-窓口の個数
 - (d) 窓口利用を作成する-窓口利用間隔
 - (e) 退店を作成する
3. リンクの作成

最初にアイテムという概念を理解して欲しい．銀行窓口モデルでは客，工場製造モデルでは製品，パケット通信モデルではパケットが，時間とともに流れていく．これらの流れていくものを抽象化したものを，S4ではアイテムと表現している（図9参照）扱うモデルに応じてアイテムを変更することで，色々なモデルを扱うことができるようになる．

この実習では銀行窓口を扱っている．銀行を窓口を訪れるのは客である．客は窓口でサービスを受けて退去していく．そこでアイテムを客と表記することにする（図10参照）．

銀行には客が来店してくる．その来店客を生成することを設定する．来店は客が生成されることであるから生成部品を活用する．そのため仮にパケット通信モデルではパケットを生成することになる．シミュレーションでは客は確率的に発生する．今回は指数分布に従うものとする（図11参照）．その頻度は平均60秒間に1人と設定することにする（図12参照）．

銀行窓口モデルにおいて入店した客は窓口へ移動する．なお窓口の前にはサービスを待つ客が待機するスペースがあることをイメージして欲しい．窓口は一般的用語でサーバー（Server）あるいはファシリティ（Facility, 処理施設）とも表現する（図13参照）．

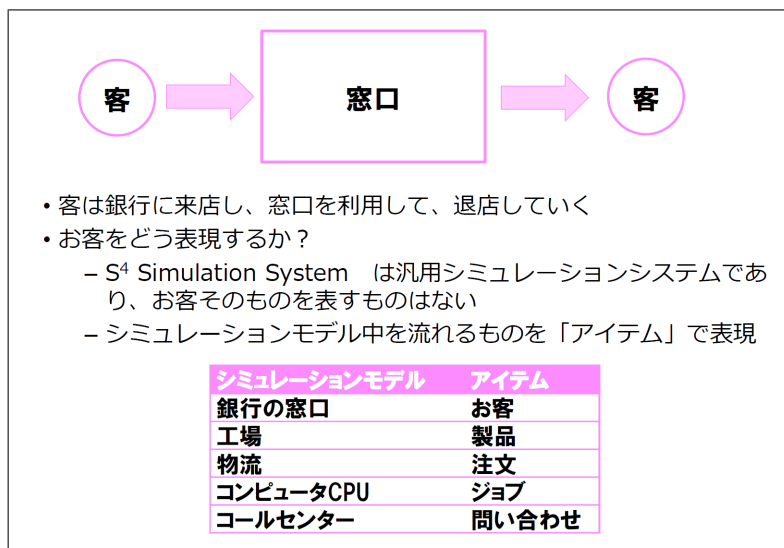


図 9: 客とアイテムの関係

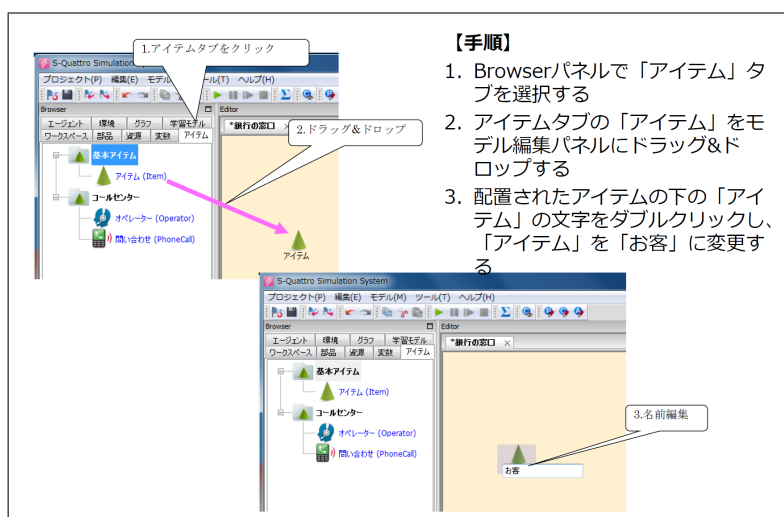


図 10: アイテムとしての客の生成

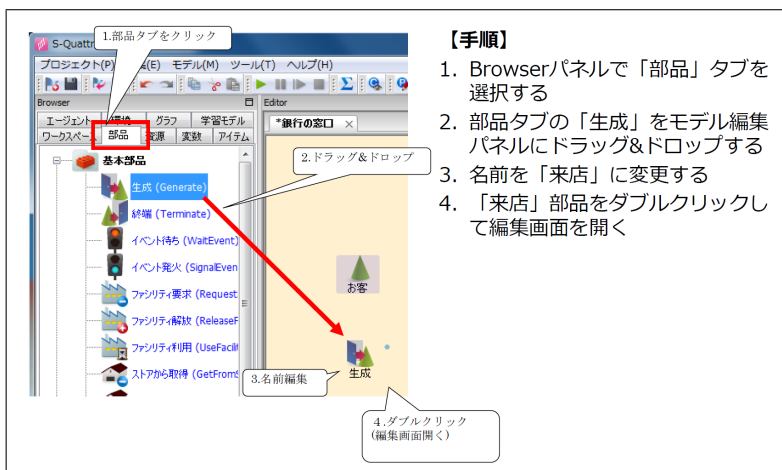


図 11: 客の生成と来店 その 1（生成元の設定）

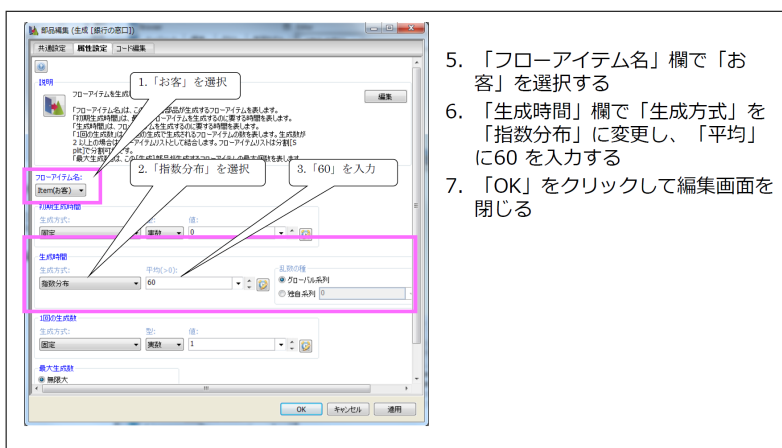


図 12: 客の生成と来店 その 2（到着間隔の設定）

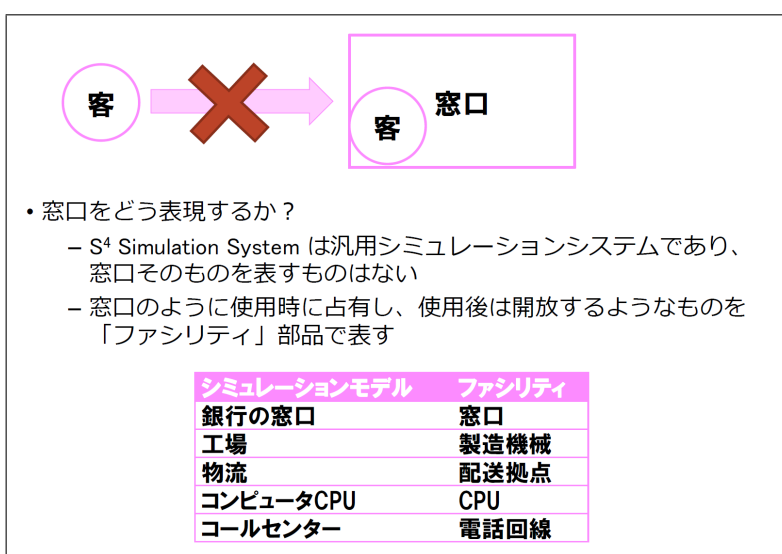


図 13: ファシリティの設置

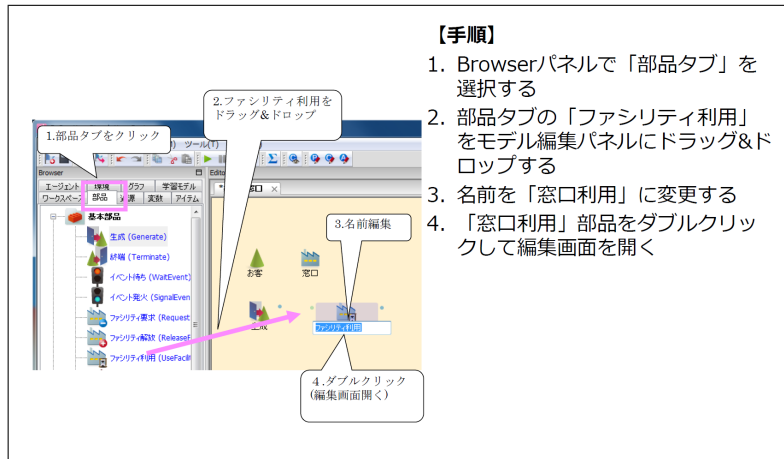


図 14: ファシリティの利用設定（名前の変更）

客は窓口を使うため、部品からファシリティ利用を選択し、その名前を「窓口利用」として用語を統一する（図 14 参照）..

ファシリティリスト欄から窓口を選択し、その設定を行う（図 15 参照）. 今回は客のサービス時間は指数分布に従い、その平均処理時間 50 人とする. また列の先頭から一人ずつ処理することから一人を選択する. また窓口に並んでいる時間（待ち時間）を計測しておくため利用の記録でモニターを選択する.

◇【注意】：窓口が複数あるときはどうするか、サービス時間の分布を変えるときはどうするか、考えておこう.

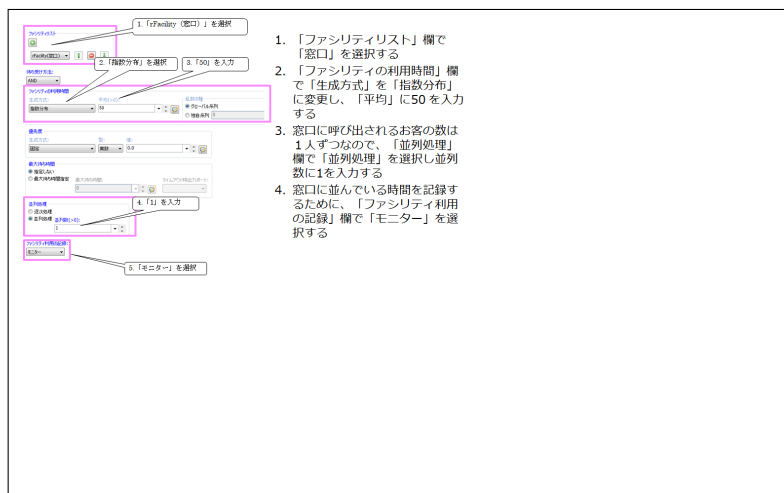


図 15: ファシリティの利用設定（サービス時間など）

今回の銀行モデルでは待つことができるスペースは 30 としているため 30 人と設定する（図 16 参照）. また統計データを記憶するように設定しておく.

◇【注意】：待ちスペースを変更するときはどうするか、考えておこう.

銀行モデルの窓口においてサービスを受けた客は施設を離れるものとする. 客の離れることを客の退去とも呼ぶが、入店に対応させて「退店」とする.「退店」した後の客の動向までは分析しないため、「終端」と解釈もできる（17 参照）.

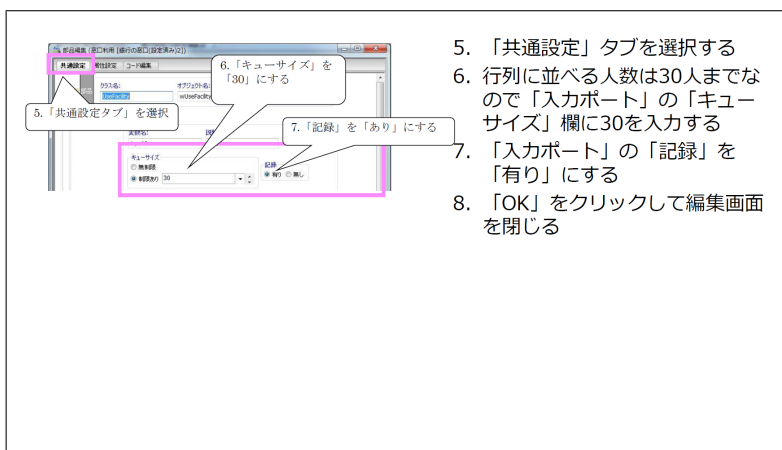


図 16: ファシリティの利用設定（待ちスペースなど）

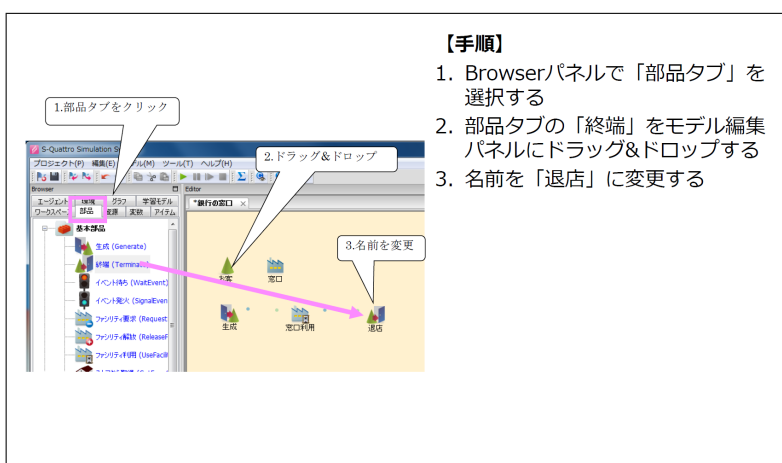


図 17: 客の退店（終端処理，退去）

客の流れていく経路(リンク)を作成する。「来店」と「窓口利用」,「窓口利用」と「退店」を接続する(18 参照)。

◇:【注意】画面上の「お客」,「窓口」には経路を作成しない。その理由を考えておこう。

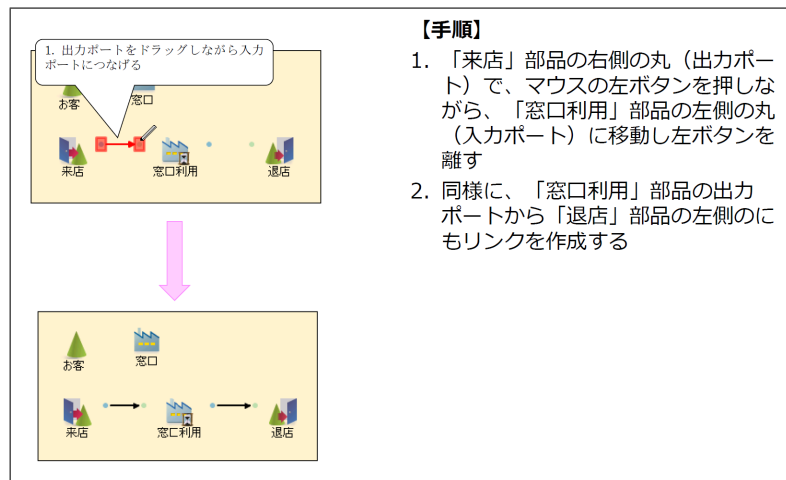


図 18: 客の退去(終端処理)

6.3 モデルパラメータの設定とシミュレーションの実行

ここでは次のことを行う。

1. モデルパラメータを設定する
 - (a) シミュレーション時間設定
 - (b) 最適化
 - (c) 実験計画
2. モデルを実行する
3. 結果を見る・可視化する
 - (a) 結果サマリ
 - (b) ヒストグラム
 - (c) 窓口のパフォーマンス可視化

最初はシミュレーション時間を設定する(図 19 参照)。この時間に比例してサイコロを振る回数が増加することになる。例えばサイコロを振る実験において 1 の目が出る確率は理想的には 1/6 となるが、確率が 1/6 になるためには、6 回では不十分である。100 回でも難しいかもしれない。そのためこの例では、シミュレーションを繰り返す回数を 100000 時間としている。つまり 100000 時間、銀行の窓口を観測することになる。計算機でシミュレーションをすることのメリットは乱数に再現性があることである。乱数の種、シーズの値を一定にすれば、乱数といいながらも同じパターンで出現する仕様になっている。この再現性を利用すれば、行員数、行員のスキルなどに応じて、平均待ち時間、平均行列長などを検証することが可能となる。

上記で設定した時間、シミュレーションを実施する(図 20 参照)。

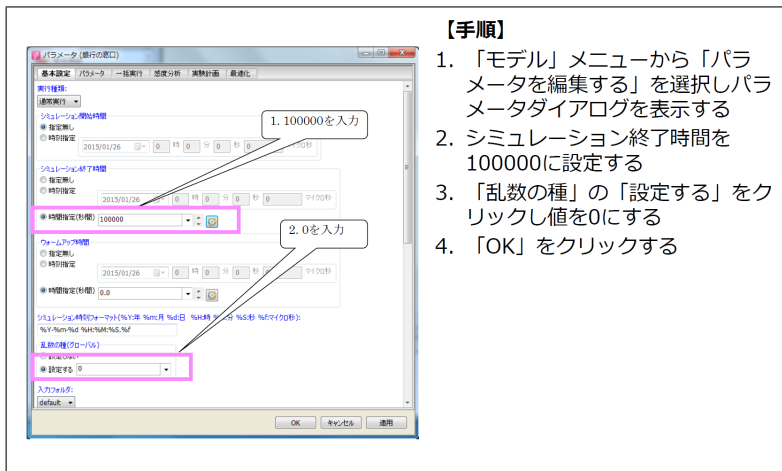


図 19: モデルパラメータの設定（シミュレーション環境の設定）

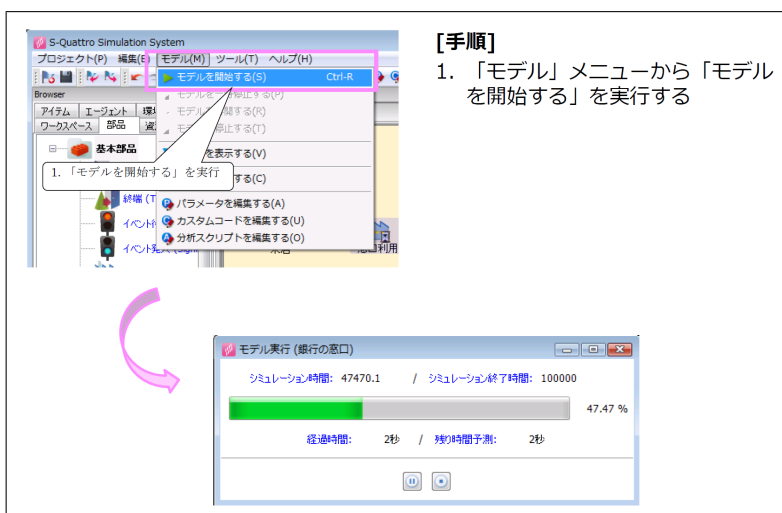


図 20: モデルの実行（シミュレーションの実施）

S4などシミュレータを活用すると、自動的にシミュレーション結果を処理して、様々な統計データをまとめてくれる。この機能をサマリー機能と呼ぶ。上記で設定した時間、シミュレーションを実施する（図21参照）。

◇：【注意】今回の実験ではたまたまS4を利用している。S4以外のシミュレーションツールを利用する場合も、これまで説明したことと同様のことができると考えて欲しい。そのように考えることで、柔軟に新しいプログラミング言語などを学習することができる。

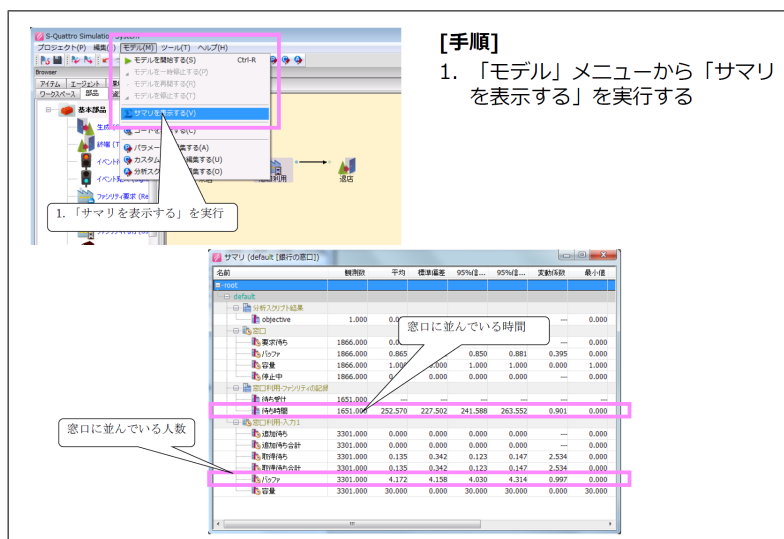


図 21: サマリーの表示（シミュレーション結果の分析）

サマリーはヒストグラムなどのグラフとして出力もできる（図22参照）。グラフなどにより可視化すると、窓口の混雑状況などが理解しやすくなる。

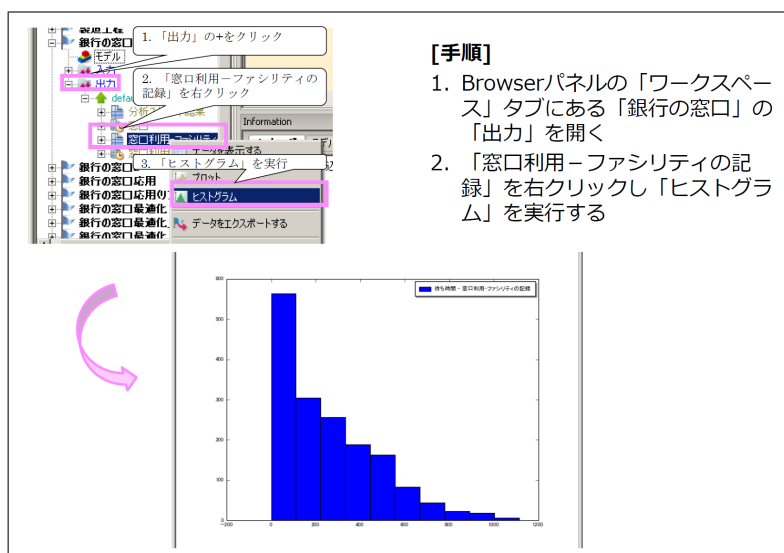


図 22: 結果の可視化（シミュレーション結果のグラフ化）

6.4 様々な検証

シミュレーションではただモデルを構築し実行結果が得られただけで満足してはいけない。パラメータを変更する意味を考えた上で、パラメータを変更し、その効果を検証することが重要になる。なおパラメータを変更することは実社会では、時間・経済的コストが発生するが、シミュレーションではそのコストが発生しないという特徴がある。ここではこれまでに設定したパラメータを変更し、その効果を待ち時間の値を比較するなどして検証する。

6.4.1 ベテラン行員

ベテラン行員は作業に熟達していることから、一般的に処理時間が短くなることが予想される。そのため「窓口利用」部品をダブルクリックし編集画面を開いて、「ファシリティの利用時間」欄で「平均」に30を入力する（図 23 参照）。最初の結果と比較してみる。

The image shows a screenshot of a simulation software interface. The top part displays the 'Facility Utilization Time' settings for 'Window Utilization'. A pink box highlights the 'Average' field, which is set to 30. A callout points to this field with the text '平均を30にする'. Below this, a pink arrow points to the 'Simulation Results' table. The table has columns for 'Name', 'Count', 'Average', 'Standard Deviation', '95% Confidence Interval', '95% Prediction Interval', 'Waiting Time', and 'Cost'. The table shows various simulation results, including 'Window Utilization' and 'Waiting Time'. A callout points to the 'Waiting Time' column with the text '窓口で並んでいる時間'. Another callout points to the 'Waiting Time' column with the text '窓口で並んでいる人数'.

【手順】

1. 「窓口利用」部品をダブルクリックし編集画面を開く
2. 「ファシリティの利用時間」欄で「平均」に30を入力する
3. 「OK」をクリックし編集画面を閉じる
4. 「モデル」メニューを開き「モデルを開始する」を実行する

図 23: ベテランの銀行員にする（サービス時間の短縮）

6.4.2 行員数を増やす

一人の行員では行列が長くなってしまふ。そこで行員数を1から2以上に変更してみる（図 24 参照）。最初の結果と比較してみる。行員数を増やすと待ち時間が減少するはずである。増やしすぎても人件費などのコストが増加する。対費用効果を検証してみることも必要である。数値結果をまとめて整理すると良い（図 25 参照）。

【手順】

1. 「窓口」部品をダブルクリックし編集画面を開く
2. 編集画面で「同時利用容量」を2にする
3. 「OK」をクリックし編集画面を閉じる
4. 「窓口利用」部品をダブルクリックし編集画面を開く
5. 「ファシリティの利用時間」欄で「平均」に50を入力する
6. 「並列処理」欄で「並列数」を2にする
7. 「OK」をクリックし編集画面を閉じる
8. 「モデル」メニューを開き「モデルを開始する」を実行する

図 24: 行員を 2 倍にする（数の増加）

条件	平均待ち行列 (人)	平均待ち時間(秒)
窓口:Aさん配置	4.172	252.57
窓口:ベテラン配置	0.505	31.502
窓口:2人配置	0.180	11.158

- ・ 2人配置するのが一番よい結果になった
- ・ どの程度パフォーマンスが向上したかを視覚化したい
- ・ 窓口のパフォーマンスはどのように定量化するか
 - 単位時間あたりに処理出来るお客数=スループット
- ・ スループットを可視化したい

図 25: 3 種類の結果を比較する（数の増加）

第III部

第2日目

7 M/M/s と M/M/s/K

前回までに学んだ S4 を利用して，M/M/1 と M/M/s の性能の比較を行う．ただしシミュレーションの条件など実験方法は各自で考えること．なお S4 を操作する前に何を考えるかを整理してから取り組むようにすること．本資料では，シミュレーションソフト”S-Quattro”で，M/M/1/K、M/M/S/K モデルの作成、結果の比較を行う。また、シミュレーションに使うシード値によって実行結果が異なる場合があるが、統計結果に大きい影響がない。

7.1 評価モデル

通信ネットワークにおける実例を例にして評価を行う。図 26 に示すように中継システムは波長多重化と時分割多重化方式があげられる。

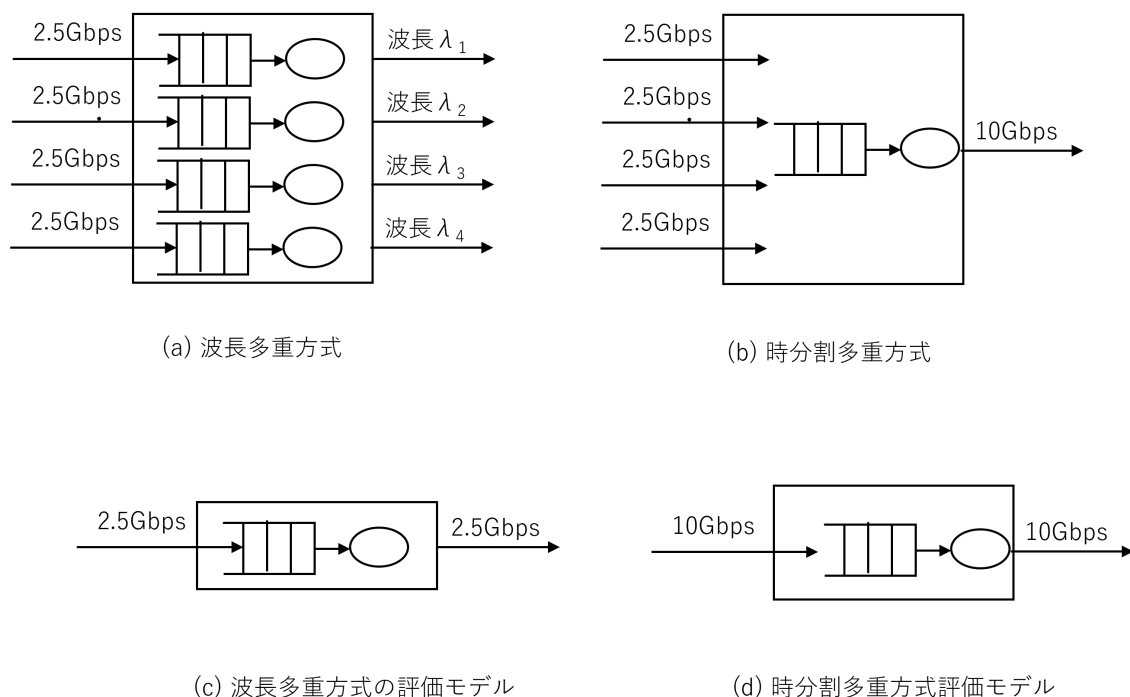


図 26: パケット多重化装置

評価モデルにすると図 26(c),(d) に示すようにモデル化できる。両方は M/M/1/K であるが、到着率と処理速度が異なる。波長多重方式の理論計算結果は図 27 に示す。

ネットワークの技術の発展とともにネットワークはますますに複雑になり、理論評価は限界がある。シミュレーションはネットワークの評価に一つの手段となる。続いてシミュレーションツール SQ4 を用いてネットワークの評価方法を説明する。

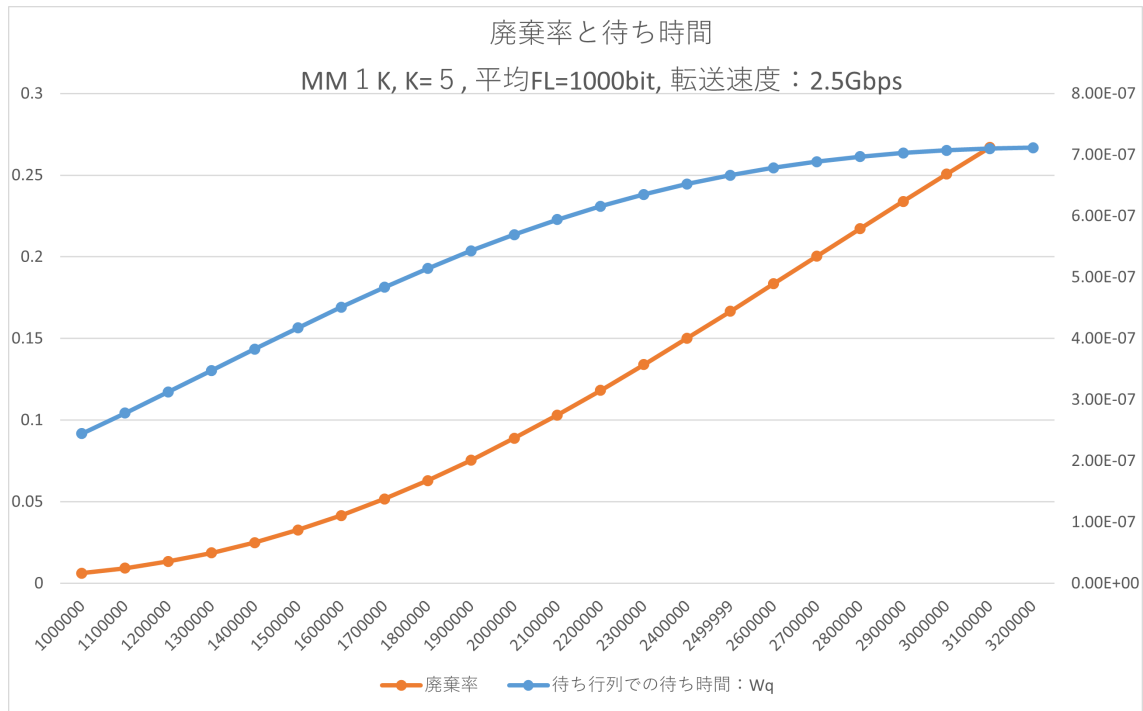


図 27: 波長多重方式の理論計算結果

7.2 M/M/1/K

7.2.1 M/M/1/K モデルの作成

M/M/1/K モデル (図 28) 作成手順は以下の通りであり、この例では K=6 とする。
SQ 4 を起動して図 29 になり、以下の通りに設定する。

1. プロジェクトをクリックして新規プロジェクトに名前を付けて作成する。例えば名前を mm 1 k にする。
2. 画面左「Browser」内の「アイテム」から「アイテム」、「資源」から「ファシリティ」を Editor にドラッグ&ドロップする。
3. 「Browser」内の「部品」から「生成」、「ファシリティ利用」、「終端」をドラッグ&ドロップする。
4. 「生成」、「ファシリティ利用」、「終端」を結ぶ。

7.2.2 パラメータの設定

図 28 にある「生成」をダブルクリックすると図 30 のようにウィンドウが開き、パラメータを設定することができる。到着のトラフィックに関する設定はここで行う。

ここでは、フローアイテム、生成時間について説明する。

1. フローアイテム欄では、「アイテム」を選択する。
2. 生成時間欄では、基本的に指数分布とする。

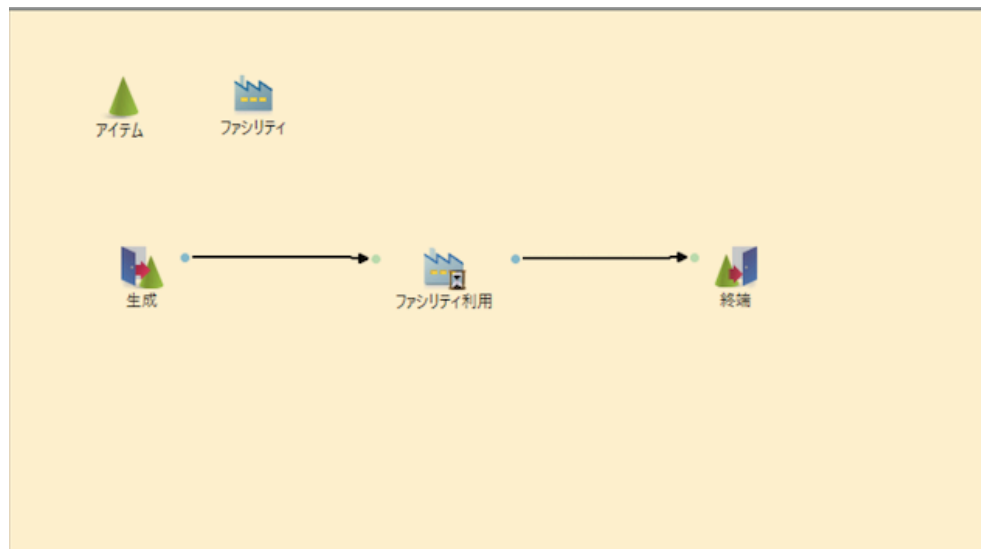


図 28: M/M/1/K モデル

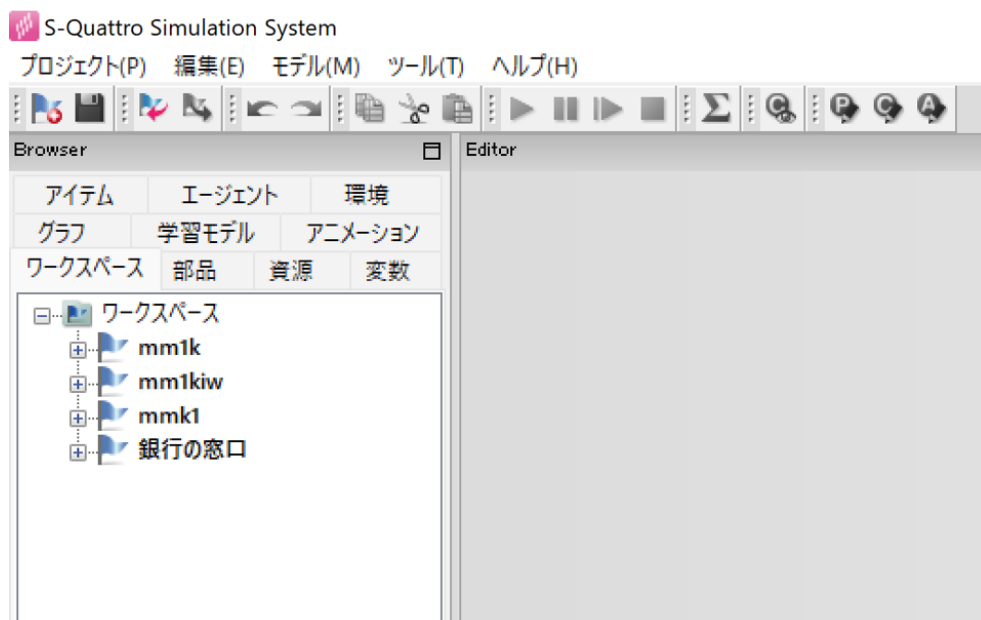


図 29: 最初画面

3. 平均生成時間がまず 60 とする。即ち時間 60 の間にパケットを 1 生成することを表す。

部品編集 (生成 [MM1])

共通設定 属性設定 コード編集

説明

フローアイテムを生成します。

「フローアイテム名」は、この【生成】部品が生成するフローアイテムを表します。
「初期生成時間」は、最初にフローアイテムを生成するのに要する時間を表します。
「生成時間」は、フローアイテムを生成するのに要する時間を表します。
「生成時間系列の生成」は生成時間系列の生成方法を表します。
「1回の生成数」は、1回の生成で生成されるフローアイテムの数を表します。生成数が2以上の場合はフローアイテムリストとして結合します。フローアイテムリストは分割[Split]で分割可能です。
「最大生成数」は、この【生成】部品が生成するフローアイテムの最大個数を表します。

編集

フローアイテム名:

Item(アイテム) ▼

初期生成時間

生成方式: 型: 値:

固定 ▼ 実数 ▼ 0 ▼

生成時間

生成方式: 平均(>0): 乱数の種

指数分布 ▼ 60 ▼

☒ グローバル系列
☐ 独自系列 0 ▼

生成時間系列の生成:

毎回生成し直す ▼

1回の生成数

生成方式: 型: 値:

固定 ▼ 実数 ▼ 1 ▼

最大生成数

☒ 無限大
☐ 指定 1 ▼

図 30: 「生成」パラメータ

次に、「ファシリティ利用」をダブルクリックすると図 31 が開き、パラメータを設定することができる。ここでは、ファシリティリスト、ファシリティ利用時間、ファシリティ利用の記録について説明する。

1. ファシリティリスト欄では、使用するファシリティ(rFacility)の選択をする。
2. ファシリティ利用時間欄では、生成方式、到着間隔を設定する。待ち行列モデルの場合、生成方式は基本的に指数分布となる。また、図 31 の場合、平均利用時間が 50 となっており、時間 50 の間、1つのパケットをファシリティ利用することを表す。
3. ファシリティ利用の記録では、時系列モニターにする。シミュレーション時間内でのファシリティ利用状況を時系列ごとに観察できる。

部品編集 (ファシリティ利用 [MM1])

共通設定 属性設定 コード編集

— はしは、記録しません。
 — "モニター"は、ファシリティの利用の可否とファシリティの利用に要した時間を記録します。
 — "時系列モニター"は、ファシリティの利用の可否とファシリティの利用に要した時間を時系列的に記録します。

ファシリティリスト

rfacility(ファシリティ) [↑] [↓] [←] [→]

待ち受け方法:
 AND

ファシリティの利用時間

生成方式: 平均(>0): 乱数の種

指数分布 50 ●グローバル系列
 ○独自系列 0

優先度

生成方式: 型: 値:

固定 実数 0.0

最大待ち時間

●指定しない
 ○最大待ち時間指定 最大待ち時間: タイムアウト時出力ポート:

0

並列処理

●逐次処理
 ○並列処理 並列数(>0):

1

ファシリティ利用の記録:

時系列モニター

図 31: 「ファシリティ利用」パラメータ

次に、図 31 にある「共通設定」をダブルクリックすると図 32 にある入力ポートに対応するキューのサイズを設定する。キューのサイズは、行列に並ぶことのできるパケット数なので、K-1 の値を入力する。図 32 の場合、キューサイズが 5 なので、M/M/1/6 モデルとなる。また、無制限とすると、M/M/1 モデルとなる。また、キューサイズの欄の右にある「記録」を「有り」にして記録を残す。

シミュレーション時間の設定は図 33 のようにすればいい。

設定できたら簡単なプロジェクトができた。それで、「モデルを開始」（緑の右向きの矢印）を押すとシミュレーションが開始する。

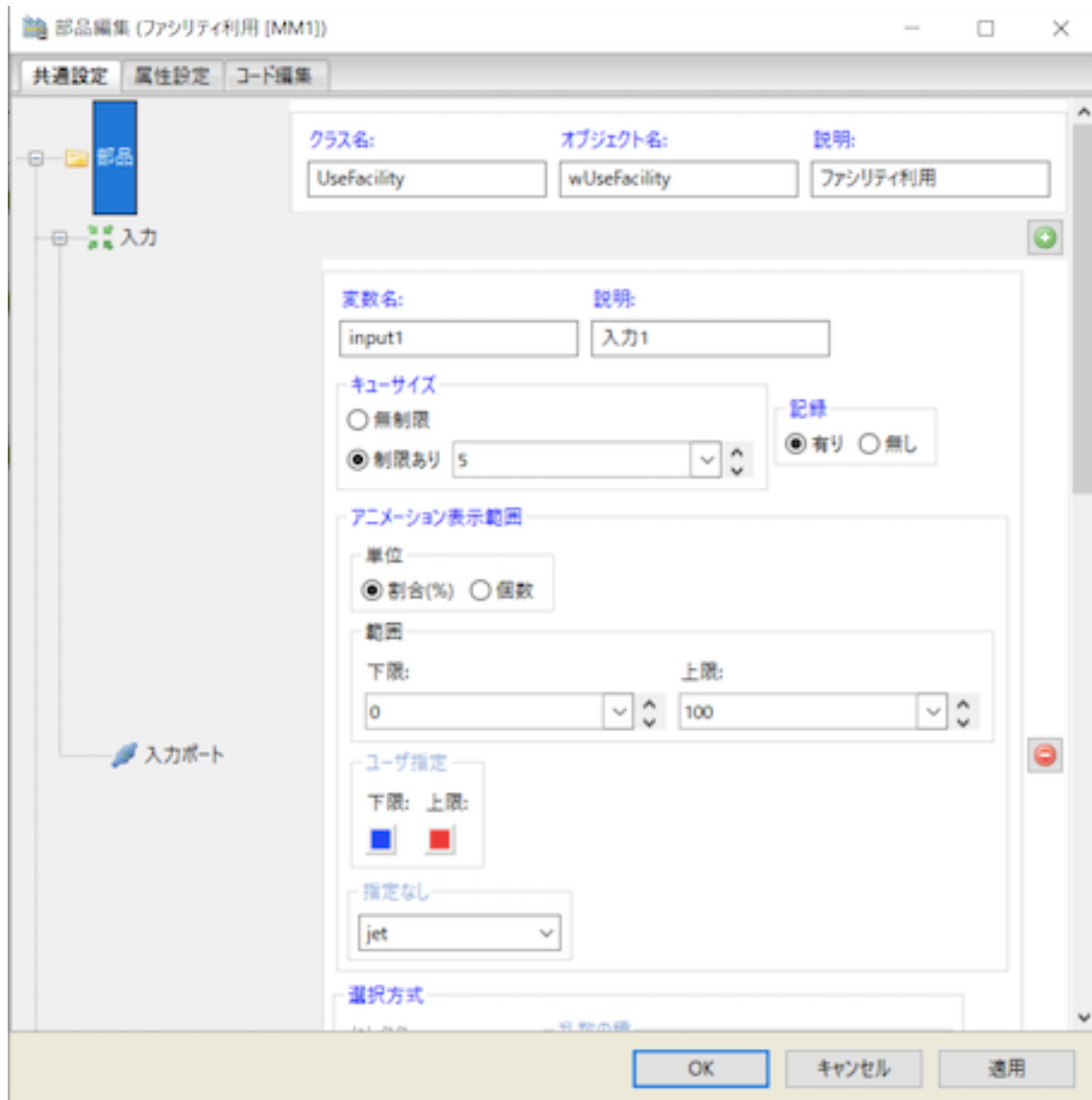


図 32: キューの設定

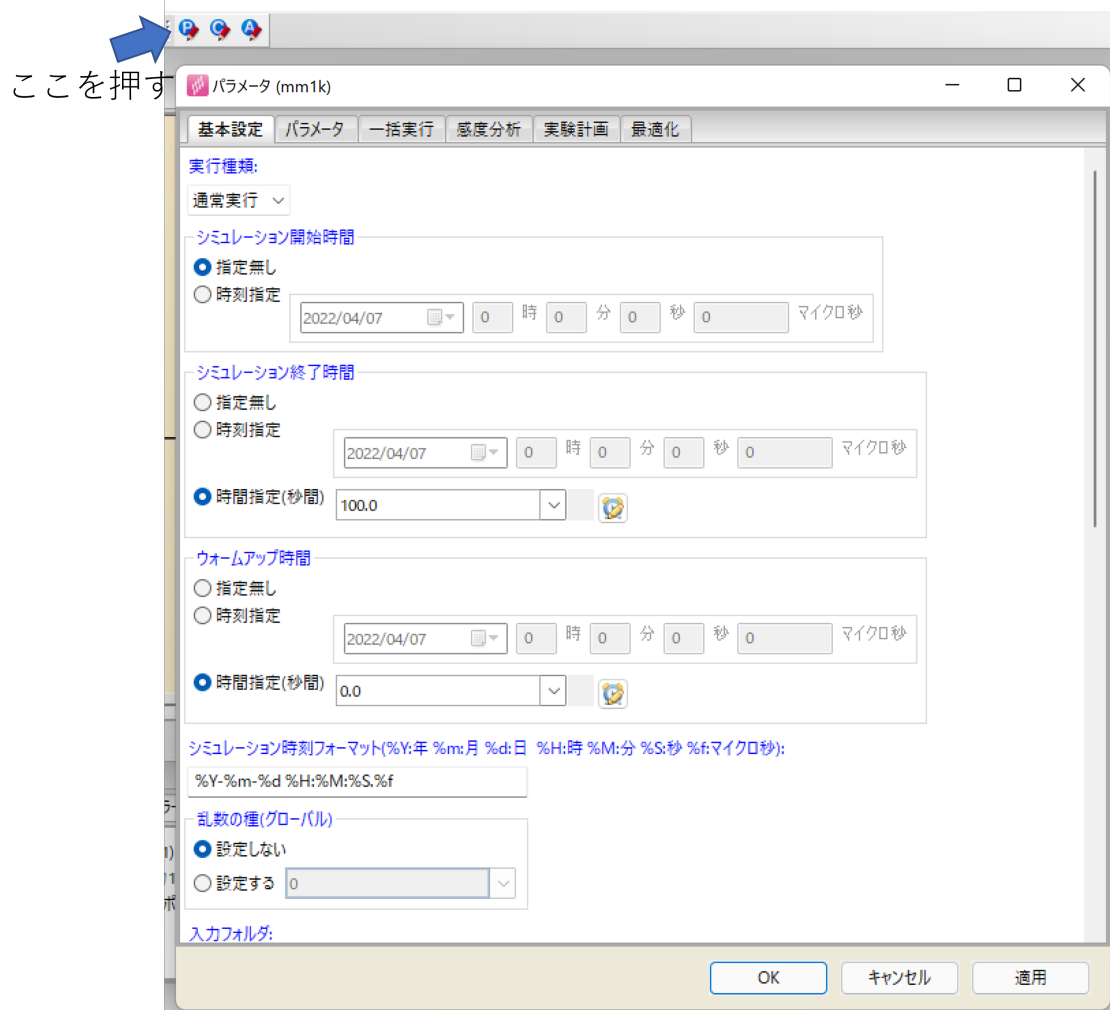


図 33: 時間

7.2.3 実行結果の表示

画面上部の「Σ」を押すと、図 34 が表示される。ここでは、平均値に着目する。「ファシリティ」の「バッファ」の平均は、ファシリティの利用率を表す。「ファシリティ利用-ファシリティ」の「待ち時間」の平均は、シミュレーション時間内での平均待ち行列時間 (利用できるまで行列に並んでいる時間の平均) を表す。

「ファシリティ利用-入力 1」の「バッファ」の平均は、平均待ち行列内客数 (利用できるまで行列に並んでいるパケット数の平均) を表す。

また、実測値はシミュレーション時間 1000000 で行なった結果である。シミュレーション時間が 1000000 を超えると、結果が出るまでに時間がかかってしまうことがあり、シミュレーション時間を 100000 以上にしても、さほど理論値と実測値の差は縮まらなかった。また、M/M/1/6 での理論値と実測値の比較は表 1 の通りである。理論値と実測値で差が生じているが、シミュレーション時間の長さによる影響を受けていると考えられる。

名前	観測数	平均	標準偏差	95%信頼...	95%信頼...	変動係数	最小値	25%値	中央値	75%値	最大値	レンジ
root												
default												
ファシリティ												
要求待ち	19260.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
バッファ	19260.000	0.787	0.410	0.781	0.792	0.521	0.000	1.000	1.000	1.000	1.000	1.000
容量	19260.000	1.000	0.000	1.000	1.000	0.000	1.000	1.000	1.000	1.000	1.000	0.000
停止中	19260.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
ファシリティ利用-ファシリティ												
待ち受け	15728.000	---	---	---	---	---	---	---	---	---	---	---
待ち時間	15728.000	93.038	111.799	91.291	94.786	1.202	0.000	0.000	51.620	151.111	891.353	891.353
ファシリティ利用-入力 1												
追加待ち	31460.000	0.066	0.248	0.063	0.068	3.775	0.000	0.000	0.000	0.000	1.000	1.000
追加待ち合計	31460.000	0.066	0.248	0.063	0.068	3.775	0.000	0.000	0.000	0.000	1.000	1.000
取得待ち	31460.000	0.213	0.410	0.209	0.218	1.920	0.000	0.000	0.000	0.000	1.000	1.000
取得待ち合計	31460.000	0.213	0.410	0.209	0.218	1.920	0.000	0.000	0.000	0.000	1.000	1.000
バッファ	31460.000	1.763	1.831	1.743	1.783	1.039	0.000	0.000	1.000	3.000	5.000	5.000
容量	31460.000	5.000	0.000	5.000	5.000	0.000	5.000	5.000	5.000	5.000	5.000	0.000
分析スクリプト結果												
objective	1.000	0.000	---	---	---	---	0.000	0.000	0.000	0.000	0.000	0.000

図 34: 実行結果

	実測値	理論値
平均待ち行列時間	93.038	91.3
平均行列内客数	1.763	1.52

表 1: 理論値と実測値の比較

7.2.4 廃棄パケットの記録

キュー一杯の状態にパケットが到着したら廃棄される。廃棄パケットの記録を取得するためにタンクを追加する。平均システム内パケット数及び廃棄率について説明する。図 35 は、図 28 から「タンク」を加え、パケットの数をカウントするモデルである。

1. 生成、ファシリティ利用と端末の結線を削除。
2. 「資源」をクリックして「タンク」を追加。

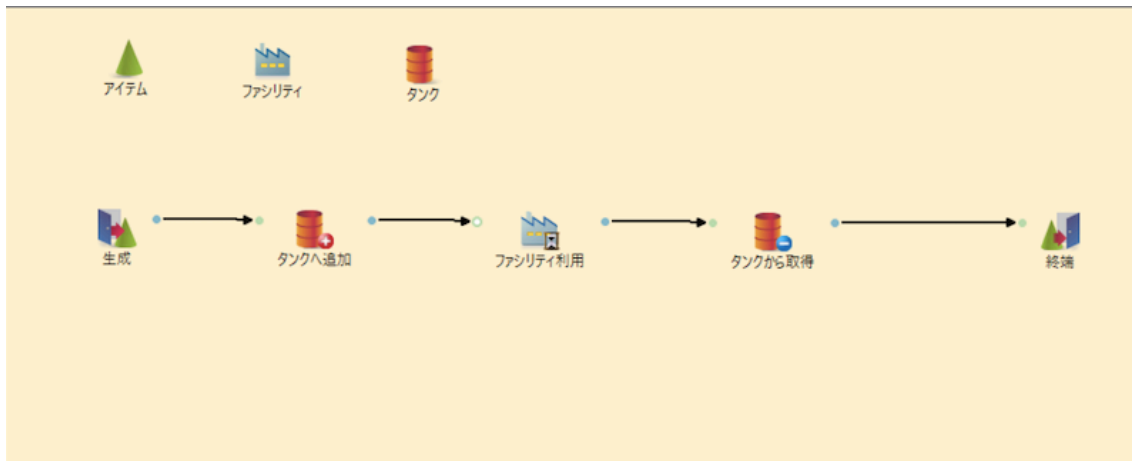


図 35: ストア付き M/M/1/K モデル

3. 部品から「タンクへの追加」を追加する。
4. 部品から「タンクから取得」を追加する。
5. 結線を追加。
6. 「タンク」（結線がない方）をダブルクリックして（容量）を設定する。例えば 1 E8
7. 「タンクへの追加」、「タンクからの取得」をそれぞれダブルクリックして「タンク」欄に「rTank」にする。

「タンクへの追加」について図 36 のように○を付けた所を注意しながら「記録」欄に「有り」と設定していい。「タンクからの取得」も同じように設定する必要がある。

「タンクへ追加」にパケットが通過すると「タンク」のカウントが+1 され、「タンクから取得」を通過するとカウントが-1 される。そうすることで、平均システム内パケット数を算出できる。廃棄率は、「タンクへ追加」と「タンクから取得」の観測数を見ることで、廃棄されたパケット数を算出できる。

図 37 を見ると、「タンク」の「バッファ」から平均システム内パケット数は 2.646、廃棄率は $1 - \frac{15951}{18116} = 0.1195$ となる。理論値と実測値の比較は表 2 の通りである。

	実測値	理論値
平均システム内パケット数	2.817	2.290
廃棄率	1.195×10^{-1}	7.7×10^{-2}

表 2: 理論値と実測値の比較

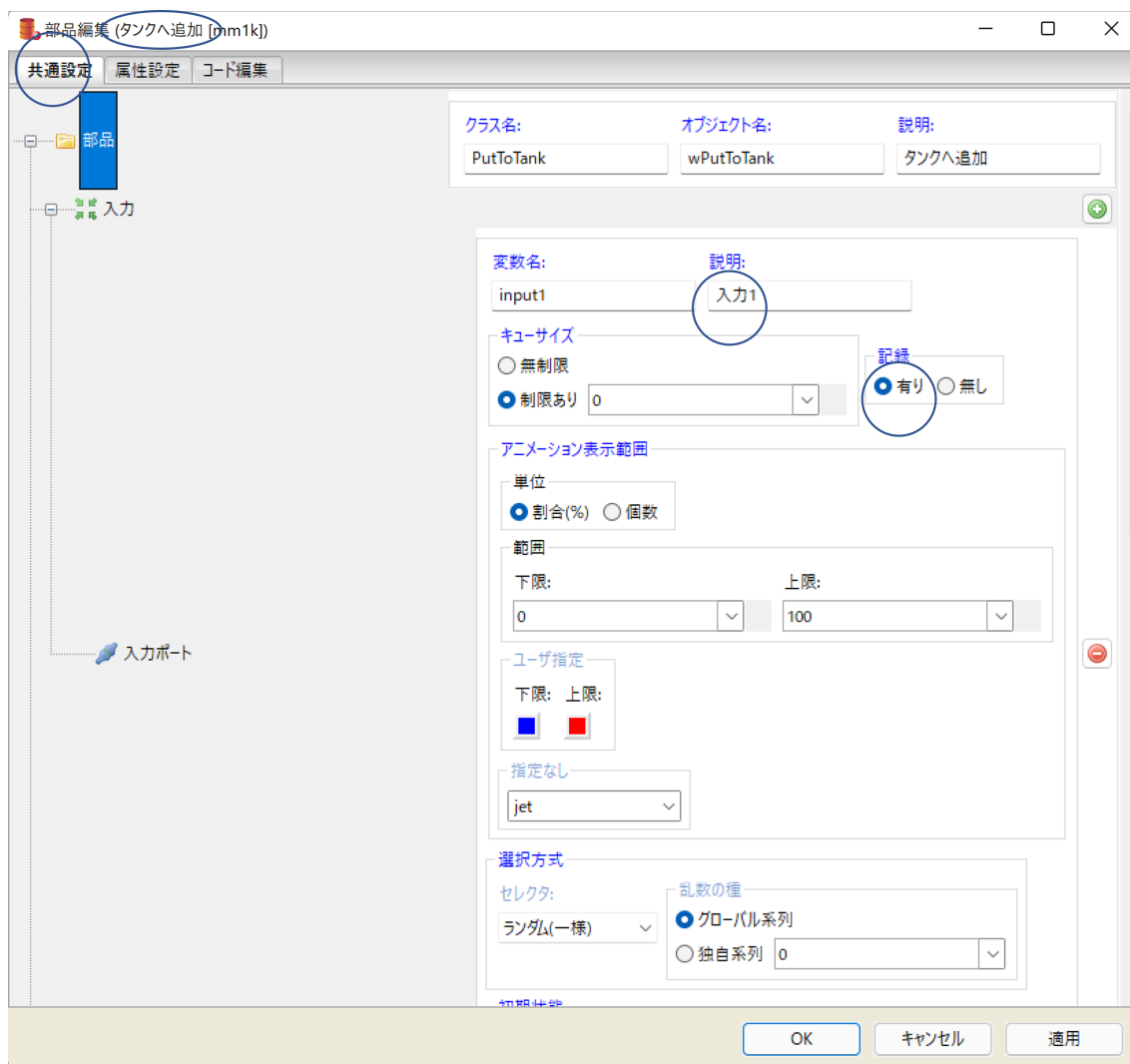


図 36: タンクの設定

名前	観測数	平均	標準偏差	95%信頼...	95%信頼...	変動係数	最小値	25%値	中央値	75%値	最大値	レンジ
default												
タンク												
追加待ち	30906.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
追加待ち合計	30906.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
取得待ち	30906.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
取得待ち合計	30906.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
バッファ	30906.000	2.817	2.334	2.791	2.843	0.829	0.000	1.000	2.000	5.000	7.000	7.000
容量	30906.000	100000000.0	0.000	100000000.0	100000000.0	0.000	100000000.0	100000000.0	100000000.0	100000000.0	100000000.0	0.000
タンクから取得-入力1												
追加待ち	15951.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
追加待ち合計	15951.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
取得待ち	15951.000	1.000	0.000	1.000	1.000	0.000	1.000	1.000	1.000	1.000	1.000	0.000
取得待ち合計	15951.000	1.000	0.000	1.000	1.000	0.000	1.000	1.000	1.000	1.000	1.000	0.000
バッファ	15951.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
容量	15951.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
タンクへ追加-入力1												
追加待ち	18116.000	0.051	0.220	0.048	0.054	4.307	0.000	0.000	0.000	0.000	1.000	1.000
追加待ち合計	18116.000	0.051	0.220	0.048	0.054	4.307	0.000	0.000	0.000	0.000	1.000	1.000
取得待ち	18116.000	0.888	0.315	0.884	0.893	0.354	0.000	1.000	1.000	1.000	1.000	1.000
取得待ち合計	18116.000	0.888	0.315	0.884	0.893	0.354	0.000	1.000	1.000	1.000	1.000	1.000
バッファ	18116.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
容量	18116.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
ファシリティ												
要求待ち	19339.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
バッファ	19339.000	0.797	0.402	0.791	0.802	0.505	0.000	1.000	1.000	1.000	1.000	1.000
容量	19339.000	1.000	0.000	1.000	1.000	0.000	1.000	1.000	1.000	1.000	1.000	0.000
停止中	19339.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
ファシリティ利用-ファシリティ												
待ち受け	15951.000	---	---	---	---	---	---	---	---	---	---	---
待ち時間	15951.000	99.786	116.451	97.979	101.593	1.167	0.000	0.000	58.287	161.004	865.403	865.403
ファシリティ利用-入力1												
追加待ち	30906.000	0.112	0.315	0.108	0.115	2.823	0.000	0.000	0.000	0.000	1.000	1.000
追加待ち合計	30906.000	0.112	0.315	0.108	0.115	2.823	0.000	0.000	0.000	0.000	1.000	1.000
取得待ち	30906.000	0.203	0.402	0.199	0.208	1.980	0.000	0.000	0.000	0.000	1.000	1.000
取得待ち合計	30906.000	0.203	0.402	0.199	0.208	1.980	0.000	0.000	0.000	0.000	1.000	1.000
バッファ	30906.000	1.909	1.917	1.887	1.930	1.005	0.000	0.000	1.000	4.000	5.000	5.000
容量	30906.000	5.000	0.000	5.000	5.000	0.000	5.000	5.000	5.000	5.000	5.000	0.000
分析スクリプト結果												
objective	1.000	0.000	---	---	---	---	0.000	0.000	0.000	0.000	0.000	0.000

図 37: 実行結果 (タンク付き)

7.3 M/M/S/K

7.3.1 M/M/S/K モデルの作成

M/M/S/K モデルの作成手順は以下の通りであり、ここでは M/M/2/7 モデルを例にして作成する。また、レイアウトは M/M/1/K モデルと変わらないため、省略する。

1. 「ファシリティ」アイテム (結線がない方) をダブルクリックすると図 38 のような画面が開くので、「同時利用容量」を S にあたる数字「2」を入力する。
2. 「ファシリティ利用」(結線画ある方) アイテムをダブルクリックすると図 39 のような画面が開くので、「並列処理」を S にあたる数字「2」を入力する。

資源編集 (ファシリティ [numberofcustomers (2)])

共通設定 属性設定

説明

排他的に利用可能な複数の離散資源を持つファシリティオブジェクトです。

「同時利用容量」は、同時に利用可能な離散資源の容量を表します。
「キュー」は、資源が利用できない場合に並ぶキューを表します。
「記録」は、資源の利用状態を記録するかどうかを表します。

編集

同時利用容量(>0):

2

キュー:

FIFO

記録

☒ 有り ☐ 無し

容量変化スケジュール

☒ スケジュール無し
☐ スケジュール有り 開始オフセット時間: 0.0

スケジュール

☒ 表指定

	時間	値
1	0.000000	1

☐ 式 [(時間1, 容量1), (時間2, 容量2), ...]: [(0, 1)]

OK キャンセル 適用

図 38: 同時利用容量の設定

共通設定

属性設定

コード編集

— は、記録しません。
 — "モニター"は、ファシリティの利用の可否とファシリティの利用に要した時間を記録します。
 — "時系列モニター"は、ファシリティの利用の可否とファシリティの利用に要した時間を時系列的に記録します。

ファシリティリスト

+

rfacility(ファシリティ)

↑

↓

待ち受け方法:

AND

ファシリティの利用時間

生成方式:

指数分布

平均(>0):

50

乱数の種

☒ グローバル系列
 ☐ 独自系列

0

優先度

生成方式:

固定

型:

実数

値:

0.0

最大待ち時間

☒ 指定しない
 ☐ 最大待ち時間指定

最大待ち時間:

0

タイムアウト時出力ポート:

並列処理

☐ 逐次処理
 ☒ 並列処理

並列数(>0):

2

ファシリティ利用の記録:

時系列モニター

図 39: 並列処理の設定

7.3.2 実行結果の表示及び理論値との比較

ここでも同様に、平均値に着目し、理論値との比較を行う。実行結果の例は図 40 に示す。

名前	観測数	平均	標準偏差	95%信頼...	95%信頼...	変動係数	最小値	25%値	中央値	75%値	最大値	レンジ
シミュレーション												
タンク												
追加待ち	33571.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
追加待ち合計	33571.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
取得待ち	33571.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
取得待ち合計	33571.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
バッファ	33571.000	1.012	1.189	0.999	1.025	1.175	0.000	0.000	1.000	1.000	8.000	8.000
容量	33571.000	100000000.0E	0.000	100000000.0E	100000000.0E	0.000	100000000.0E	100000000.0E	100000000.0E	100000000.0E	100000000.0E	0.000
タンクから取得-入力1												
追加待ち	16787.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
追加待ち合計	16787.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
取得待ち	16787.000	1.000	0.000	1.000	1.000	0.000	1.000	1.000	1.000	1.000	1.000	0.000
取得待ち合計	16787.000	1.000	0.000	1.000	1.000	0.000	1.000	1.000	1.000	1.000	1.000	0.000
バッファ	16787.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
容量	16787.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
タンクから取得-入力1												
追加待ち	16819.000	0.000	0.012	-0.000	0.000	81.168	0.000	0.000	0.000	0.000	1.000	1.000
追加待ち合計	16819.000	0.000	0.012	-0.000	0.000	81.168	0.000	0.000	0.000	0.000	1.000	1.000
取得待ち	16819.000	0.999	0.025	0.999	1.000	0.025	0.000	1.000	1.000	1.000	1.000	0.000
取得待ち合計	16819.000	0.999	0.025	0.999	1.000	0.025	0.000	1.000	1.000	1.000	1.000	0.000
バッファ	16819.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
容量	16819.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
ファシリティ												
要求待ち	29382.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
バッファ	29382.000	0.836	0.795	0.827	0.845	0.952	0.000	0.000	1.000	1.000	2.000	2.000
容量	29382.000	2.000	0.000	2.000	2.000	0.000	2.000	2.000	2.000	2.000	2.000	0.000
停止率	29382.000	0.000	0.000	0.000	0.000	---	0.000	0.000	0.000	0.000	0.000	0.000
ファシリティ利用-ファシリティ												
待ち受け	16786.000	---	---	---	---	---	---	---	---	---	---	---
待ち時間	16786.000	6.991	21.158	6.670	7.311	3.027	0.000	0.000	0.000	0.000	362.396	362.396
ファシリティ利用-入力1												
追加待ち	33571.000	0.001	0.025	0.000	0.001	39.378	0.000	0.000	0.000	0.000	1.000	1.000
追加待ち合計	33571.000	0.001	0.025	0.000	0.001	39.378	0.000	0.000	0.000	0.000	1.000	1.000
取得待ち	33571.000	1.164	0.795	1.156	1.173	0.683	0.000	1.000	1.000	2.000	2.000	0.000
取得待ち合計	33571.000	1.164	0.795	1.156	1.173	0.683	0.000	1.000	1.000	2.000	2.000	0.000
バッファ	33571.000	0.176	0.804	0.169	0.182	3.438	0.000	0.000	0.000	0.000	5.000	5.000
容量	33571.000	5.000	0.000	5.000	5.000	0.000	5.000	5.000	5.000	5.000	5.000	0.000
シミュレーション結果												
objective	1.000	0.000	---	---	---	---	0.000	0.000	0.000	0.000	0.000	0.000

図 40: 実行結果 mmsk

	実測値	理論値
平均待ち行列時間	6.991	10.0
平均行列内客数	0.176	0.167
平均システム内客数	1.012	0.99
廃棄率	1.902×10^{-3}	1.8×10^{-3}

表 3: 理論値と実測値の比較

平均システム内客数の実測値はファシリティのバッファの平均とファシリティ利用-入力1のバッファの平均との合計、すなわちタンクのバッファの平均に入る数値である。

7.4 実例を用いたシミュレーション

7.4.1 実例シミュレーションに関するパラメータ換算

図 27 に示す結果に使われたパラメータは図 41 に示される。同じ条件でシミュレーションで評価する場合に到着間隔、処理時間を計算して決める必要がある。

		mm1k				
FL	転送速度	h	λ	μ	a : 負荷	K
1000	2.50E+09	4.00E-07	1000000	2.50E+06	4.00E-01	5

図 41: 応用例パラメータ換算

1. 到着間隔= $1/\lambda$
2. 処理時間= h

7.4.2 M/M/1/K 以外の例のシミュレーション

指数分布ではない場合は理論評価が難しくなるが、シミュレーションは簡単にできる場合がある。例えば、パラメータ処理時間を設定するとき、指数分布の代わりに固定値にしたら M/D/1/K になる。実験ではいろいろなケースに挑戦して結果を比較して意味深いことが分かるかもしれない。

第IV部

第3日目

8 形態分析

前回までに学んだ S4 を利用し，複数の M/M/1，一つの M/M/s，一つの優れた M/M/1 の性能を比較し，どの形態が良いか分析を行う．ただしシミュレーションの条件など実験方法は各自で考えること．また性能だけでなく多角的にどの形態が良いのか考察をすること．図 42 に示すように三つの形態についてシミュレーションで評価する．

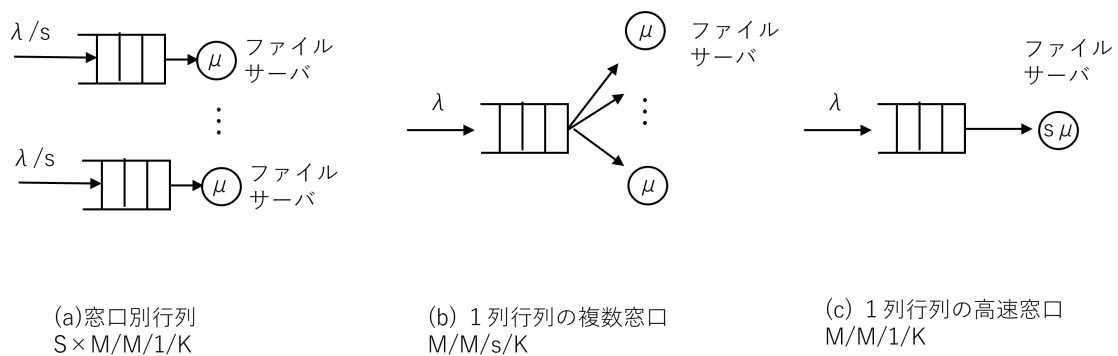


図 42: 形態比較

参考文献

- [1] 木下是雄, 『理科系の作文技術』, 中公新書 1981.
- [2] 篠田義明, 『コミュニケーション技術—実用的文章の書き方』, 中央公論社, 1986.
- [3] 杉原厚吉, 『どう書くか—理科系のための論文作法』, 共立出版, 2001.
- [4] 本多勝一, 『新装版 日本語の作文技術』, 講談社, 2005.
- [5] 阿部圭一, 『明文術 伝わる日本語の書きかた』, NTT 出版, 2006.
- [6] 酒井聡樹, 『これから論文を書く若者のために 大改訂増補版』, 共立出版, 2006
- [7] 酒井聡樹, 『これからレポート・卒論を書く若者のために』, 共立出版, 2007.
- [8] 酒井聡樹, 『これから学会発表する若者のために -ポスターと口頭のプレゼン技術-』, 共立出版, 2008.
- [9] 秋丸春夫, Rober B. Cooper, 『通信トラヒック工学』, オーム社, 1985.
- [10] 川島幸之助, 増田悦夫, 宮保憲治, 『最新コンピュータネットワーク技術の基礎』, 電気通信協会 (発売元 オーム社), 2003.
- [11] 牧野都治, 『待ち行列の応用』, 森北出版, 1969.
- [12] V.G Kulkarni, *Modeling, Analysis, Design, and Control of Stochastic Systems*, Springer (1999).
- [13] 尾崎俊治, 『確率モデル入門』, 朝倉書店, 1999.
- [14] 栗原謙三, 明石吉三, 『経営情報処理のためのオペレーションズリサーチ』, コロナ社, 2001.
- [15] 高橋幸雄, 森村英典, 『混雑と待ち』, 朝倉書店, 2001.
- [16] 滝根哲哉, 西尾章治郎, 伊藤大雄, 『インターネット <5> ネットワーク設計理論』, 岩波書店, 2003.
- [17] 高橋敬隆, 吉野秀明, 山本尚生, 戸田彰, 『わかりやすい待ち行列システム—理論と実践』, コロナ社, 2003.
- [18] 北岡正敏, 『例題でわかる待ち行列理論入門』, 日本理工出版会, 2010.
- [19] 有澤誠, 『モデルシミュレーション技法』, 共立出版, 1999.
- [20] Michael Pidd, *Computer Simulation in Management Science 4th Ed.*, New York: John Wiley & Sons, 1998.
- [21] <http://www.opnet.com>.
- [22] <http://www.isi.edu/nsnam/ns/>.
- [23] Burbank, J., “Modeling and simulation: a practical guide for network designers and developers [Guest Editorial],” *IEEE Communications Magazine*, vol.47, no.3, pp.118-173, 2009.

- [24] Hasan M.S., Yu H., Carrington A., Yang T.C., “Co-simulation of wireless networked control systems over mobile ad hoc network using SIMULINK and OPNET”, *IET Communications*, vol.3, no.8, pp.1297-1310, 2009.
- [25] Dimitri P. Bertsekas, “Traffic Behavior and Queuing in a QoS Environment”, *Proceedings of OPNETWORK2004*, Session 1813 2004.
- [26] <https://www.msi.co.jp/s4/>
- [27] Kevin Brown, Leann Christianson, *Data and Computer Communications and Computer Networking with internet Protocols and Technology: : Opnet Lab Manual to Accompany the Seventh edition*, Upper Saddle River, NJ: Prentice Hall, 2004.
- [28] Leann Christianson, Kevin Brown, *Business Data Communication: Opnet Lab Manual to Accompany the fifth edition*, Upper Saddle River, NJ: Prentice Hall, 2004.
- [29] 松下 温, 重野 寛 『無線 LAN 技術講座』, ソフト・リサーチセンター 1994.
- [30] 阪田 史郎, 『無線 LAN』, オーム社, 2004.
- [31] 国井 利泰, 『LAN 概説』, 共立出版, 1995.
- [32] IEEE Std. 『IEEE Std. 802.11 (1999). Wireless LAN medium access control (MAC) and physical layer (PHY) specifications. Reference number ISO/IEC 8802-11, 1999(E)』, IEEE, 1999.